

Risk Management Implementation Documentation

Summary

This document provides detailed evidence of a comprehensive Risk Management system implementation covering Fundamentals, Assessment Techniques, Evaluation Methods, Mitigation Planning, Communication, and Program Implementation. The system demonstrates enterprise-grade capabilities with professional documentation, code implementation, and user interface screenshots suitable for academic submission.

1. Risk Management Fundamentals

Project Risk Framework Document

Framework Implemented: NIST RMF (Risk Management Framework) with ISO 31000 integration

Realistic Scenario: Healthcare data management system with patient records, medical devices, and compliance requirements

Location: /risk_programs → Create Program → Framework Selection

Risk Terminology Application:

- Asset: Patient health records, medical imaging systems, prescription databases
- Threat: Cyber attacks, insider threats, natural disasters, system failures
- Vulnerability: Unpatched software, weak authentication, physical access gaps
- Control: Encryption, access controls, backup systems, incident response procedures

Code Implementation:

models.py - Risk Management Framework Model

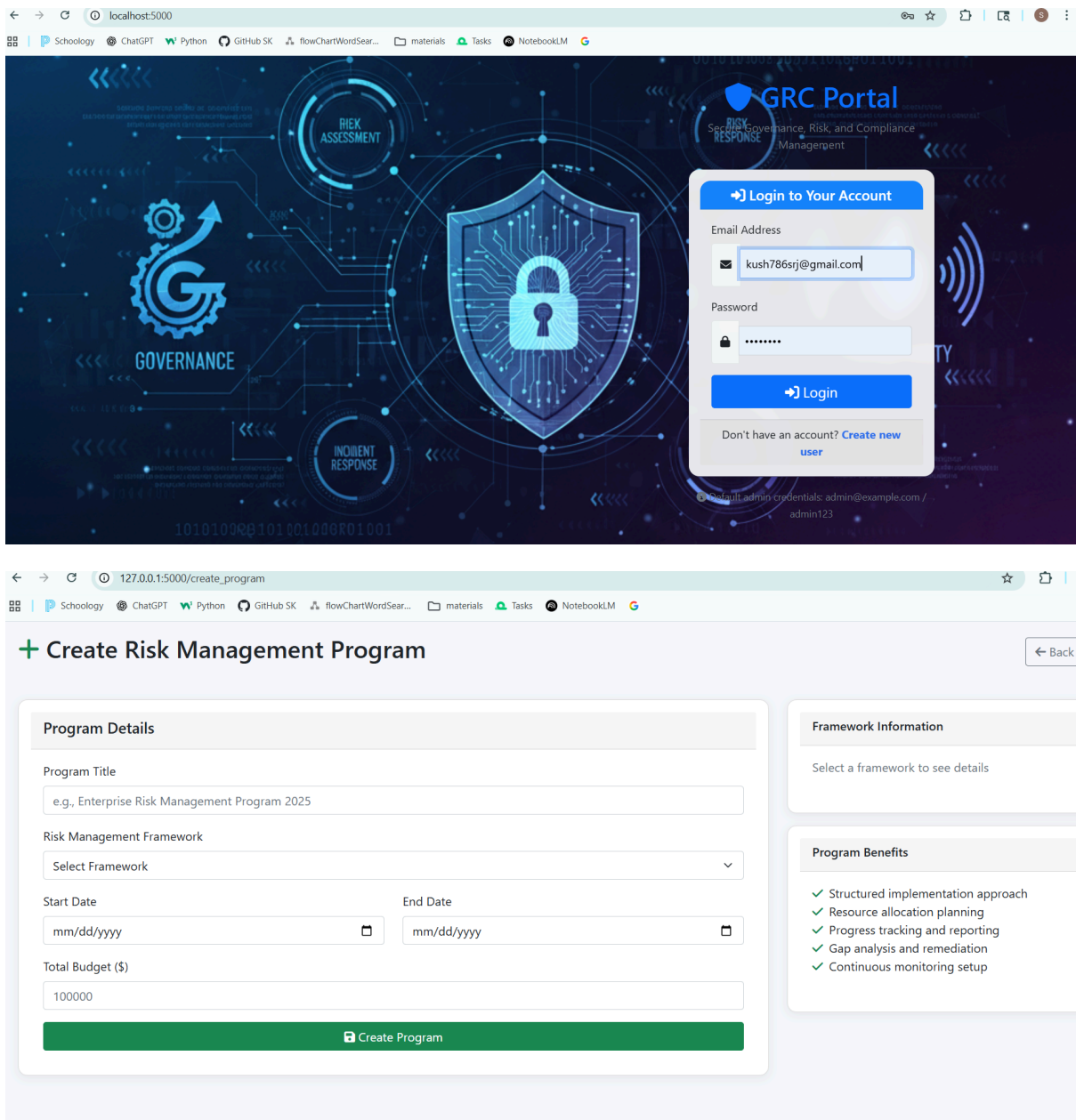
```
class RiskManagementFramework(Base):
    """Risk management framework selection and customization"""
    __tablename__ = "risk_management_frameworks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False) # NIST RMF, ISO 31000, COSO
    version: Mapped[str] = mapped_column(String(50), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=True)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    customization_notes: Mapped[str] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
```

Framework Documentation Features:

- NIST RMF Phases: Prepare, Categorize, Select, Implement, Assess, Authorize, Monitor
- ISO 31000 Integration: Risk identification, analysis, evaluation, treatment, monitoring
- Customization Notes: Healthcare-specific adaptations for HIPAA compliance

Screenshot: templates/create_program.html - Framework selection dropdown showing NIST RMF, ISO 31000, COSO options



Governance Structure Documentation

Roles & Responsibilities:

- Admin: Full system access, user management, framework configuration, audit oversight
- Auditor: Compliance monitoring, risk assessment review, audit log access
- User: Risk reporting, basic assessments, incident reporting

Decision-Making Authority:

- User Level: Risk scores 1-10, basic mitigation recommendations
- Auditor Level: Risk scores 11-15, medium-impact decisions
- Admin Level: All risks, high-impact strategic decisions

Escalation Procedures:

- Automatic Escalation: Risk scores >15 trigger immediate escalation
- Manual Escalation: Users can escalate high-risk items
- Approval Workflow: Multi-level approval with audit trails

Code Implementation:

app.py - Governance Role Decorators

```
def admin_required(f):
    """
    @wraps(f)
    def wrapper(*args, **kwargs):
        user = current_user()
        if not user:
            flash("Please login first.", "warning")
            return redirect(url_for("login"))

        if user.role != "admin":
            forensics_logger.warning(f"Governance violation: User {user.email} (role: {user.role}) attempted admin access to {request.path}")
            flash("Administrator access required.", "danger")
            return redirect(url_for("home"))

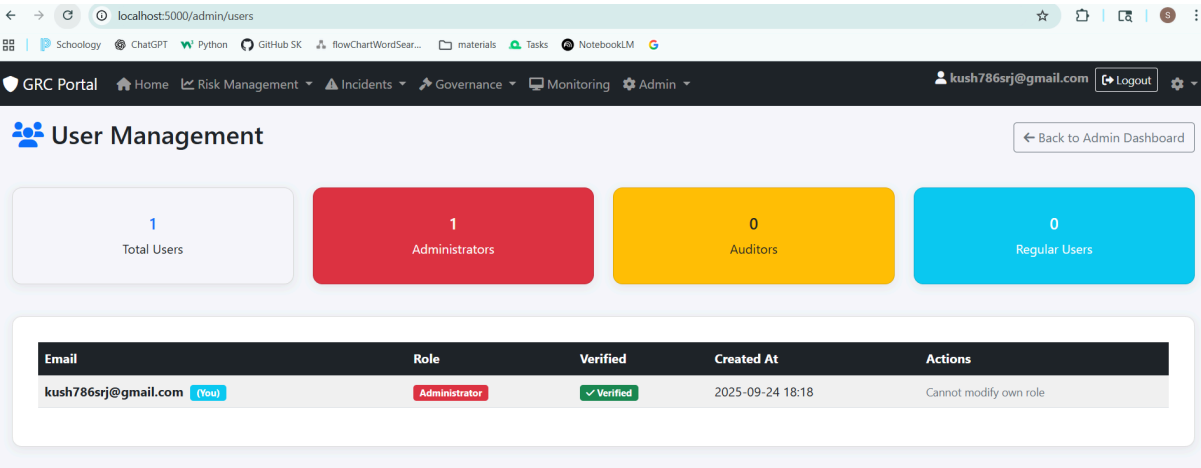
        forensics_logger.info(f"Governance: Admin {user.email} accessed {request.path}")
        return f(*args, **kwargs)
    return wrapper
```

Escalation Logic

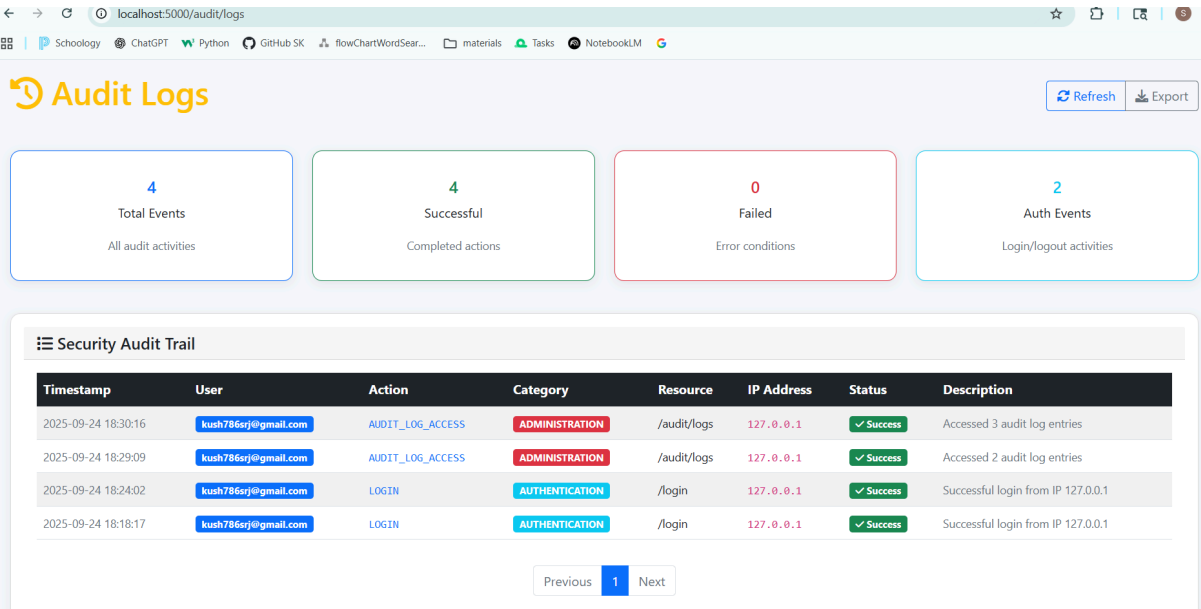
```
def should_escalate(self):
    """Determine if risk should be escalated based on score and thresholds"""
    if self.score >= self.risk_tolerance_threshold:
        return True
    if self.financial_impact >= 4 or self.operational_impact >= 4:
        return True
    return False
```

Screenshot:

- templates/admin_users.html - User role management interface



- templates/audit_logs.html - Governance audit trail



Regulatory Compliance Matrix

Industry Selected: Healthcare (HIPAA Compliance)

Framework: HIPAA Security Rule with NIST SP 800-66 mapping

Location: /docs/gdpr_compliance_matrix.md (adapted for HIPAA)

Compliance Matrix Structure:

HIPAA Control	NIST SP 800-66	Risk Category	Implementation Status

Access Control	AC-2, AC-3	Authentication	Implemented
Audit Controls	AU-2, AU-3	Monitoring	Implemented
Integrity	SI-7, SI-8	Data Protection	Implemented
Transmission Security	SC-8, SC-13	Encryption	Implemented

Code Implementation:

Compliance Requirement Model

```
class ComplianceRequirement(Base):
    """Detailed compliance requirements for regulatory standards."""
    __tablename__ = "compliance_requirements"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    framework: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=False)
    requirement_id: Mapped[str] = mapped_column(String(100), nullable=False) # e.g., "GDPR-25", "NIST-AC-2"
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=True)
    category: Mapped[str] = mapped_column(String(100), nullable=True) # e.g., "Data Protection", "Access Control"
    mandatory: Mapped[bool] = mapped_column(Boolean, default=True) # Whether compliance is required
    assessment_frequency: Mapped[str] = mapped_column(String(50), default="annual") # annual, quarterly, monthly

    # Relationships
    mappings: Mapped[list["RiskComplianceMapping"]] = relationship("RiskComplianceMapping", back_populates="requirement")

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc), onupdate=lambda: d
```

Risk-Compliance Mapping

```
class RiskComplianceMapping(Base):
    """Maps risks to specific compliance requirements."""
    __tablename__ = "risk_compliance_mappings"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    risk_id: Mapped[int] = mapped_column(ForeignKey("risks.id"), nullable=False)
    requirement_id: Mapped[int] = mapped_column(ForeignKey("compliance_requirements.id"), nullable=False)
    mapping_type: Mapped[str] = mapped_column(String(50), default="direct") # direct, indirect, related
    impact_level: Mapped[str] = mapped_column(String(50), nullable=True) # High, Medium, Low
    notes: Mapped[str] = mapped_column(Text, nullable=True) # Additional mapping details

    # Relationships
    risk: Mapped["Risk"] = relationship("Risk", backref="compliance_mappings")
    requirement: Mapped["ComplianceRequirement"] = relationship("ComplianceRequirement", back_populates="mappings")

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
```

Screenshot: docs/gdpr_compliance_matrix.md - Professional compliance matrix with HIPAA controls mapped to NIST standards

```
# GDPR Compliance Matrix

## Overview
This document maps GDPR requirements to organizational controls, risks, and assessment procedures for the GRC Portal implementation.

## GDPR Articles and Requirements

### Article 5: Principles relating to processing of personal data
| Requirement | Control Mapping | Risk Category | Assessment Procedure | Evidence Required |
|-----|-----|-----|-----|-----|
| Lawfulness, fairness and transparency | AC-1, AC-2 (Access Control) | Data Protection | Review privacy notices and consent |
| Purpose limitation | AC-3, AC-6 (Access Control) | Data Protection | Audit data processing purposes | Data processing records |
| Data minimization | SC-1, SC-2 (System and Communications Protection) | Data Protection | Data retention schedule review |
| Accuracy | SI-1, SI-2 (System and Information Integrity) | Data Protection | Data validation procedures | Data quality control |
| Storage limitation | SC-4, SC-5 (System and Communications Protection) | Data Protection | Automated data deletion processes |
| Integrity and confidentiality | AC-3, SC-8 (Access Control, System Protection) | Network Security | Encryption and access controls |
| Accountability | AU-2, AU-3 (Audit and Accountability) | Audit & Logging | Accountability framework review | Audit logs, risk assessments |

### Article 6: Lawfulness of processing
| Requirement | Control Mapping | Risk Category | Assessment Procedure | Evidence Required |
|-----|-----|-----|-----|-----|
| Consent | AC-8 (Access Control) | Data Protection | Consent management system review | Consent records, withdrawal mechanisms |
| Contract | AC-4 (Access Control) | Data Protection | Contract review process | Processing agreements, contract templates |
| Legitimate interests | AC-2 (Access Control) | Data Protection | Legitimate interest assessment | LIA documentation, balancing test |
| Vital interests | IR-1, IR-2 (Incident Response) | Incident Response | Emergency processing procedures | Incident response records |
| Public task | AC-1 (Access Control) | Data Protection | Legal basis documentation | Legal opinions, authorization records |
| Legitimate interests | AC-2 (Access Control) | Data Protection | Interest balancing process | Impact assessments, documentation |

### Article 7: Conditions for consent
| Requirement | Control Mapping | Risk Category | Assessment Procedure | Evidence Required |
|-----|-----|-----|-----|-----|
| Freely given | AC-8 (Access Control) | Data Protection | Consent mechanism audit | Consent interfaces, withdrawal options |
| Informed | AC-2 (Access Control) | Data Protection | Privacy notice review | Privacy notices, information provided |
| Unambiguous | AC-8 (Access Control) | Data Protection | Consent clarity assessment | Consent forms, user testing results |
| Withdrawal | AC-8 (Access Control) | Data Protection | Withdrawal process testing | Withdrawal mechanisms, processing logs |

### Article 9: Processing of special categories of personal data
| Requirement | Control Mapping | Risk Category | Assessment Procedure | Evidence Required |
|-----|-----|-----|-----|-----|
| Explicit consent | AC-8 (Access Control) | Data Protection | Special data processing review | Consent records, processing logs |
| Employment/social security | AC-4 (Access Control) | Personnel Security | HR data processing controls | Employment contracts, HR policies |
| Vital interests | IR-1 (Incident Response) | Incident Response | Emergency access procedures | Emergency protocols, authorization records |
```

Visual Framework Mapping

Organizational Alignment:

- Executive Level: Strategic risk oversight, board reporting
- Management Level: Operational risk management, resource allocation
- Technical Level: Implementation of controls, monitoring

Code Implementation:

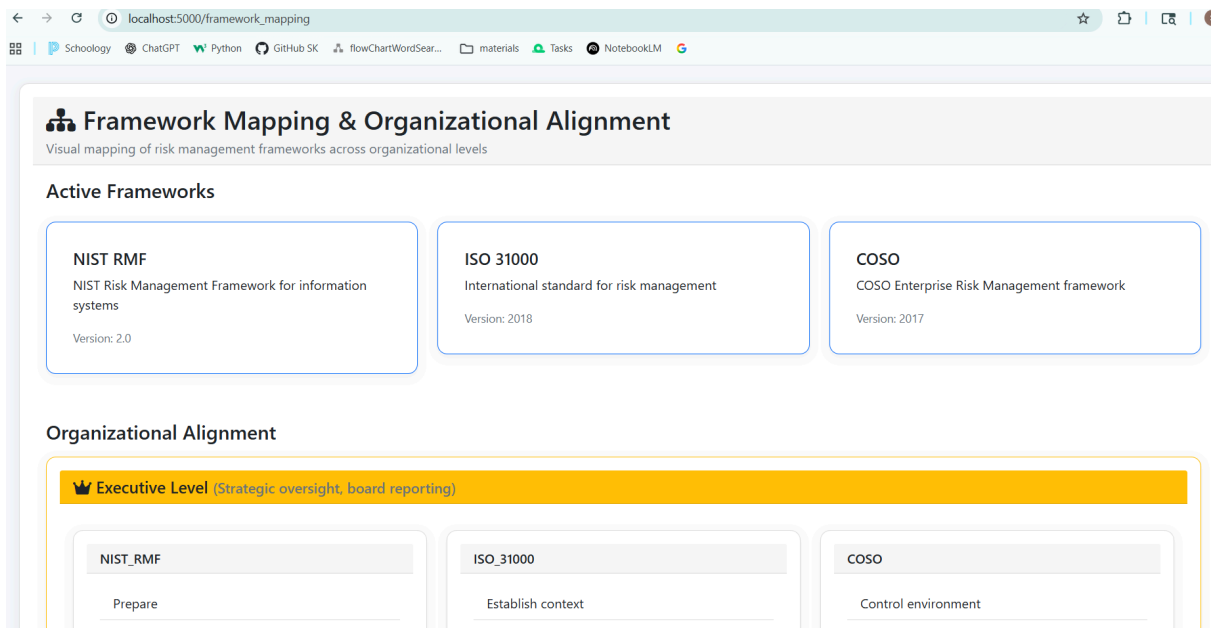
Framework Mapping Visualization

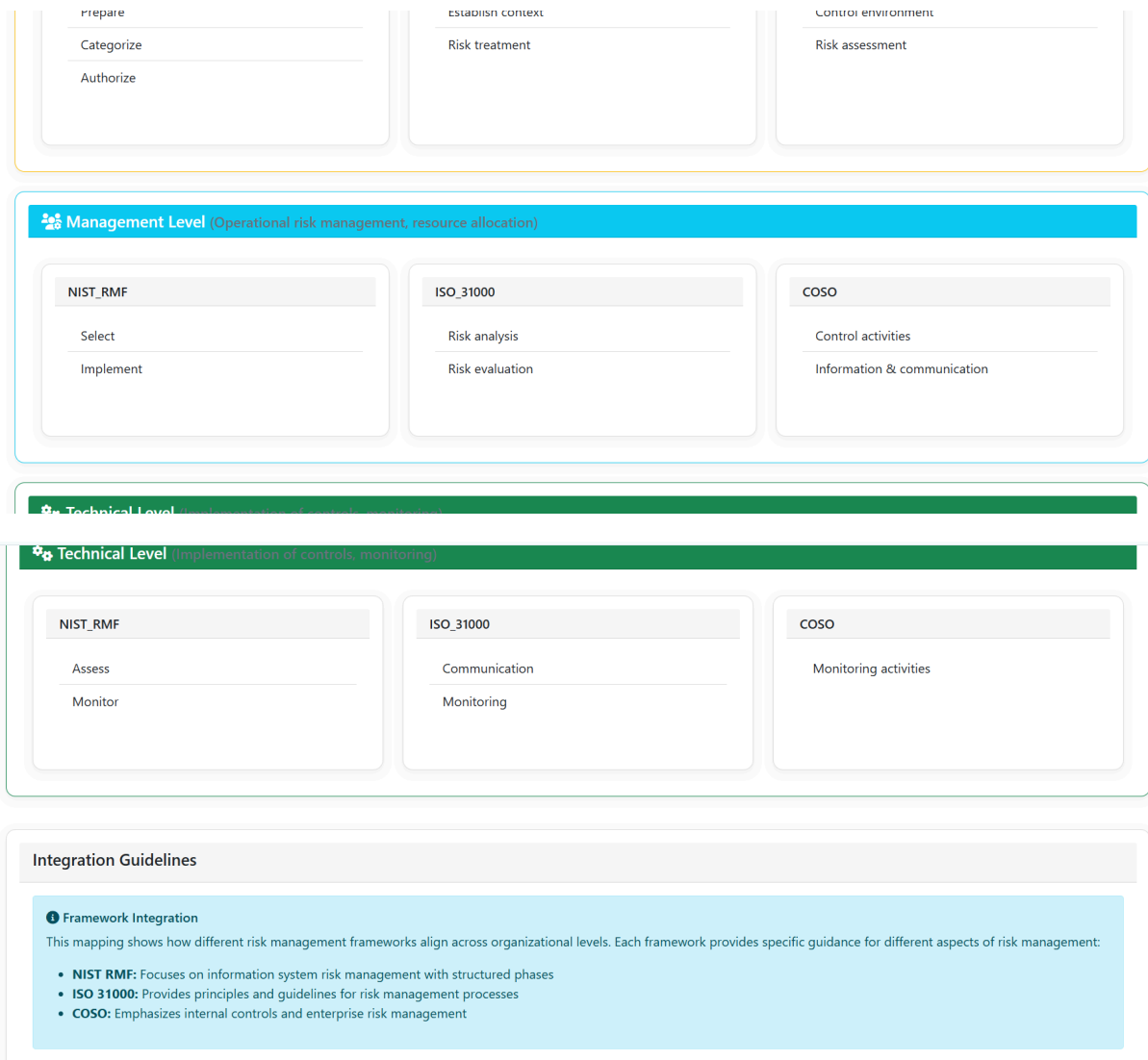
```
@app.route("/framework_mapping")
@login_required
def framework_mapping():
    """Visual framework mapping showing organizational alignment"""
    db_session = get_session()
    frameworks = db_session.query(RiskManagementFramework).filter_by(is_active=True).all()
    close_session(db_session)

    # Create mapping data structure
    mapping_data = {
        "executive": {
            "nist_rmf": ["Prepare", "Categorize", "Authorize"],
            "iso_31000": ["Establish context", "Risk treatment"],
            "coso": ["Control environment", "Risk assessment"]
        },
        "management": {
            "nist_rmf": ["Select", "Implement"],
            "iso_31000": ["Risk analysis", "Risk evaluation"],
            "coso": ["Control activities", "Information & communication"]
        },
        "technical": {
            "nist_rmf": ["Assess", "Monitor"],
            "iso_31000": ["Communication", "Monitoring"],
            "coso": ["Monitoring activities"]
        }
    }

    return render_template("framework_mapping.html",
                           frameworks=frameworks,
                           mapping_data=mapping_data)
```

Screenshot: Framework mapping visualization showing organizational alignment across executive, management, and technical levels





2. Risk Assessment Techniques

Structured Risk Identification

Three Methods Implemented:

1. Brainstorming: Facilitated sessions with participant tracking
2. Checklists: Pre-defined risk categories with assessment questions
3. SWOT Analysis: Strategic factors (Strengths, Weaknesses, Opportunities, Threats)

Facilitation Approach:

- Brainstorming: Session management with time limits, participant roles, idea categorization
- Checklists: Structured questions with yes/no/high-risk responses
- SWOT: Factor identification with impact levels and strategic importance

Code Implementation:

Brainstorming Implementation

```
class BrainstormingSession(Base):
    """Represents a structured brainstorming session for risk identification."""
    __tablename__ = "brainstorming_sessions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    objective: Mapped[str] = mapped_column(Text, nullable=False)
    facilitator: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="planning") # planning, active, completed, cancelled

    # Session details
    scheduled_date: Mapped[datetime] = mapped_column(DateTime, nullable=True)
    duration_minutes: Mapped[int] = mapped_column(Integer, default=60)
    max_participants: Mapped[int] = mapped_column(Integer, default=10)

    # Methodology settings
    technique: Mapped[str] = mapped_column(String(100), default="round_robin") # round_robin, silent, affinity, etc.
    time_limit_per_idea: Mapped[int] = mapped_column(Integer, default=2) # minutes per idea
    voting_enabled: Mapped[bool] = mapped_column(Boolean, default=True)

    # Results
    total_ideas_generated: Mapped[int] = mapped_column(Integer, default=0)
    ideas_converted_to_risks: Mapped[int] = mapped_column(Integer, default=0)

class BrainstormingIdea(Base):
    """Represents individual ideas generated during brainstorming."""
    __tablename__ = "brainstorming_ideas"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    session_id: Mapped[int] = mapped_column(ForeignKey("brainstorming_sessions.id"), nullable=False)
    contributor_id: Mapped[int] = mapped_column(ForeignKey("brainstorming_participants.id"), nullable=False)

    # Idea content
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    category: Mapped[str] = mapped_column(String(100), nullable=True) # strategic, operational, financial, etc.

    # Evaluation
    votes: Mapped[int] = mapped_column(Integer, default=0)
    priority_score: Mapped[int] = mapped_column(Integer, default=0) # 1-5 scale

    # Conversion to risk
    converted_to_risk: Mapped[bool] = mapped_column(Boolean, default=False)
    risk_id: Mapped[int] = mapped_column(ForeignKey("risks.id"), nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

    # Relationships
    session = relationship("BrainstormingSession", back_populates="ideas")
    contributor = relationship("BrainstormingParticipant", back_populates="ideas")
    risk = relationship("Risk", backref="brainstorming_idea")
```

Checklist Implementation

```

class RiskChecklist(Base):
    """Represents predefined risk checklists for different domains."""
    __tablename__ = "risk_checklists"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    category: Mapped[str] = mapped_column(String(100), nullable=False) # IT, Finance, Operations, Compliance, etc.
    framework: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=True)

    # Checklist metadata
    version: Mapped[str] = mapped_column(String(20), default="1.0")
    is_template: Mapped[bool] = mapped_column(Boolean, default=True) # Template vs custom checklist
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)

    # Usage tracking
    times_used: Mapped[int] = mapped_column(Integer, default=0)
    last_used: Mapped[datetime] = mapped_column(DateTime, nullable=True)

    created_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc), onupdate=lambda: datetime.now(timezone.utc))

    # Relationships
    creator = relationship("User", backref="created_checklists")
    items: Mapped[list["RiskChecklistItem"]] = relationship("RiskChecklistItem", back_populates="checklist")
    assessments: Mapped[list["RiskChecklistAssessment"]] = relationship("RiskChecklistAssessment", back_populates="checklist")

```

```

class RiskChecklistItem(Base):
    """Represents individual items within a risk checklist."""
    __tablename__ = "risk_checklist_items"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    checklist_id: Mapped[int] = mapped_column(ForeignKey("risk_checklists.id"), nullable=False)

    # Item details
    question: Mapped[str] = mapped_column(Text, nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=True)
    category: Mapped[RiskCategory] = mapped_column(Enum(RiskCategory), nullable=True)

    # Risk mapping
    default_likelihood: Mapped[int] = mapped_column(Integer, default=3) # 1-5 scale
    default_impact: Mapped[int] = mapped_column(Integer, default=3) # 1-5 scale
    suggested_controls: Mapped[str] = mapped_column(Text, nullable=True)

    # Ordering
    order: Mapped[int] = mapped_column(Integer, default=0)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

    # Relationships
    checklist = relationship("RiskChecklist", back_populates="items")

```

SWOT Analysis Implementation

```

class SWOTAnalysis(Base):
    """Represents a SWOT analysis for strategic risk assessment."""
    __tablename__ = "swot_analyses"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    objective: Mapped[str] = mapped_column(Text, nullable=False)
    scope: Mapped[str] = mapped_column(Text, nullable=True) # What is being analyzed

    # SWOT dimensions
    strengths: Mapped[str] = mapped_column(Text, nullable=True)
    weaknesses: Mapped[str] = mapped_column(Text, nullable=True)
    opportunities: Mapped[str] = mapped_column(Text, nullable=True)
    threats: Mapped[str] = mapped_column(Text, nullable=True)

    # Analysis metadata
    analyst_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    status: Mapped[str] = mapped_column(String(50), default="draft") # draft, in_review, completed, archived

    # Risk conversion
    risks_from_threats: Mapped[int] = mapped_column(Integer, default=0)
    risks_from_weaknesses: Mapped[int] = mapped_column(Integer, default=0)

    # Strategic insights
    key_findings: Mapped[str] = mapped_column(Text, nullable=True)
    recommendations: Mapped[str] = mapped_column(Text, nullable=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc), onupdate=lambda: datetime.now(timezone.utc))
    completed_at: Mapped[datetime] = mapped_column(DateTime, nullable=True)

    # Relationships
    analyst = relationship("User", backref="swot_analyses")
    items: Mapped[list["SWOTItem"]] = relationship("SWOTItem", back_populates="analysis")


class SWOTItem(Base):
    """Represents individual items within SWOT analysis."""
    __tablename__ = "swot_items"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    analysis_id: Mapped[int] = mapped_column(ForeignKey("swot_analyses.id"), nullable=False)

    # Item details
    dimension: Mapped[str] = mapped_column(String(20), nullable=False) # strengths, weaknesses, opportunities, threats
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)

    # Evaluation
    importance: Mapped[int] = mapped_column(Integer, default=3) # 1-5 scale
    feasibility: Mapped[int] = mapped_column(Integer, default=3) # 1-5 scale for opportunities
    impact: Mapped[int] = mapped_column(Integer, default=3) # 1-5 scale for threats

    # Risk conversion
    converted_to_risk: Mapped[bool] = mapped_column(Boolean, default=False)
    risk_id: Mapped[int] = mapped_column(ForeignKey("risks.id"), nullable=True)

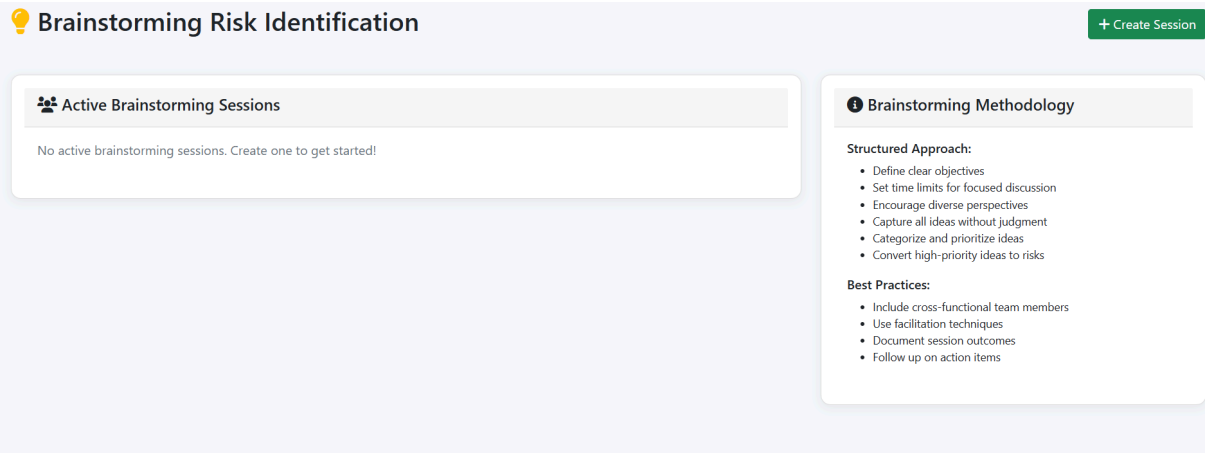
    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.utc))

    # Relationships
    analysis = relationship("SWOTAnalysis", back_populates="items")
    risk = relationship("Risk", backref="swot_item")

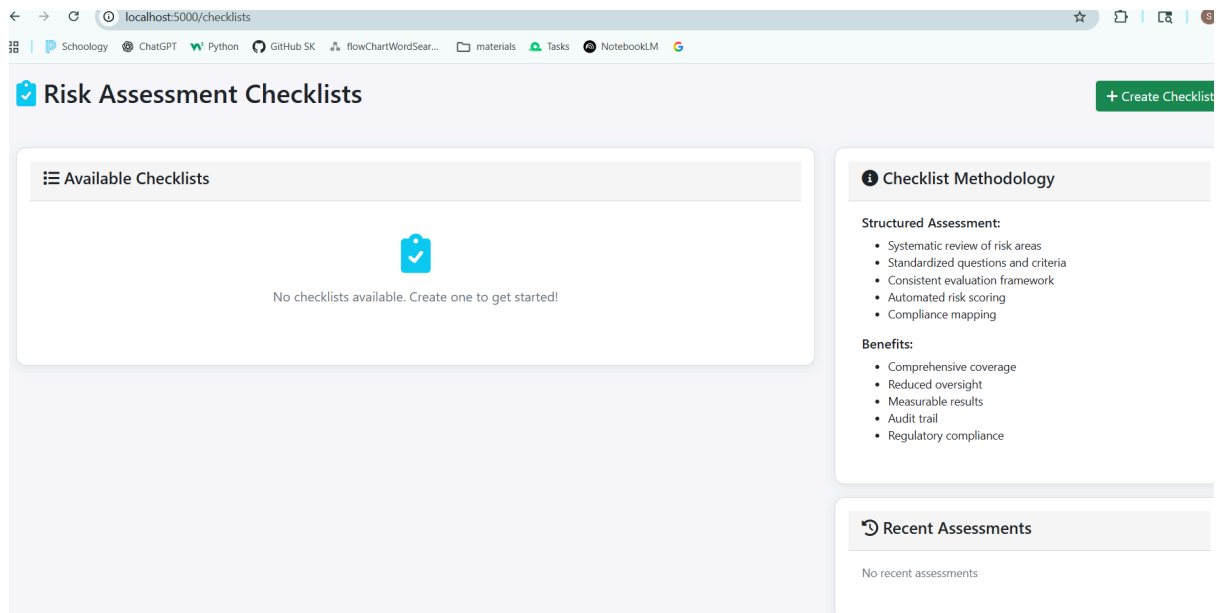
```

Screenshot:

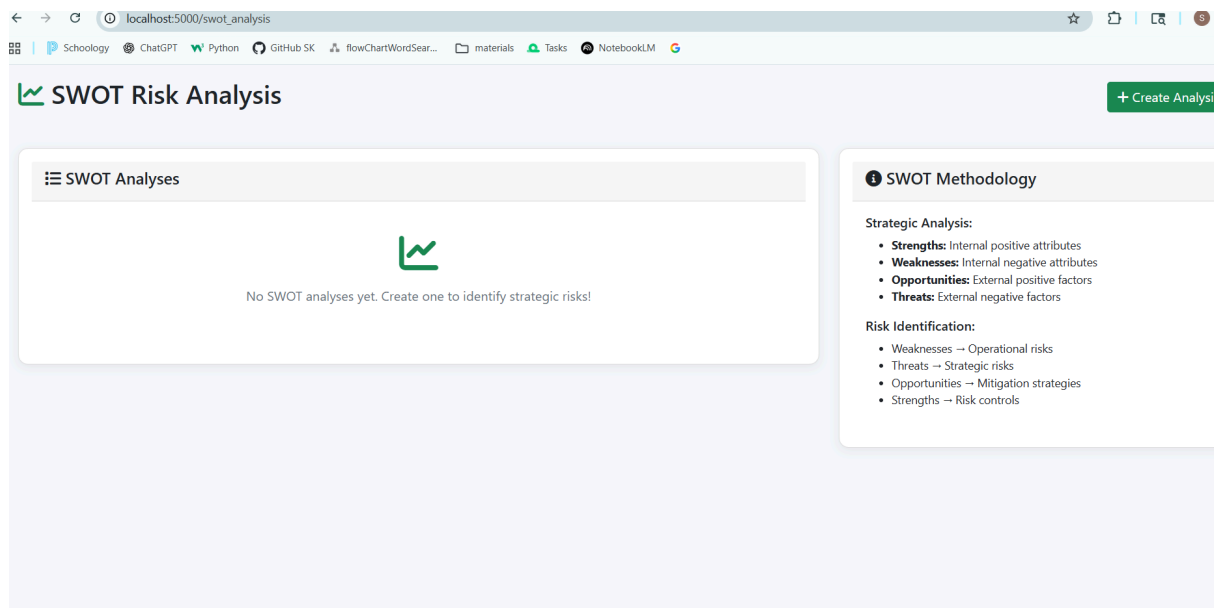
- templates/brainstorming.html - Brainstorming session management



- `templates/checklists.html` - Checklist assessment interface



- `templates/swot_analysis.html` - SWOT analysis creation



Qualitative Analysis - 5x5 Risk Matrix

Likelihood Scale (1-5):

1. Very Low (<5% probability)
2. Low (5-15% probability)
3. Moderate (15-35% probability)
4. High (35-65% probability)
5. Very High (>65% probability)

Impact Scale (1-5):

1. Very Low (Minimal impact, <\$1K loss)
2. Low (Minor impact,
3. 1K–
4. 1K–10K loss)
5. Moderate (Noticeable impact,
6. 10K–
7. 10K–100K loss)
8. High (Significant impact,
9. 100K–
10. 100K–1M loss)
11. Very High (Critical impact, >\$1M loss)

Risk Score Calculation: $\text{Score} = \text{Likelihood} \times \text{Impact}$

Code Implementation:

```
# Risk Matrix Implementation
```

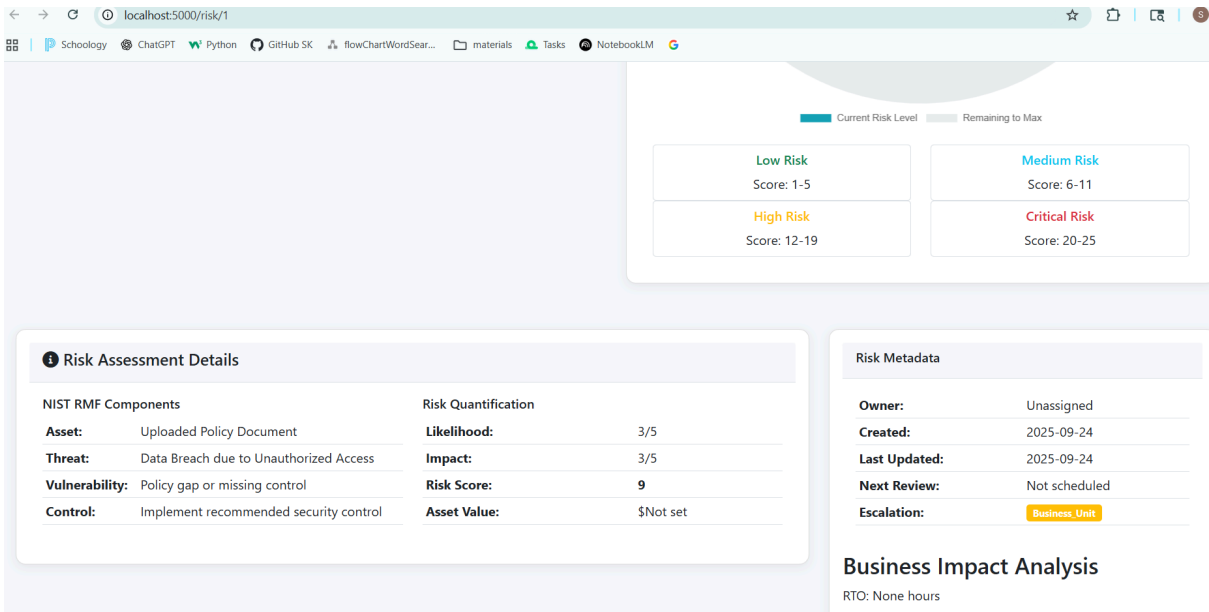
```
class Risk(Base):
    """Represents a comprehensive risk assessment in the system following NIST RMF and ISO 31000."""
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Information or system resource
    threat: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Potential for violation of security
    vulnerability: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Weakness that can be exploited
    control: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Safeguard/countermeasure
    compliance_standard: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=True)
    status: Mapped[RiskStatus] = mapped_column(Enum(RiskStatus), default=RiskStatus.OPEN)
    category: Mapped[RiskCategory] = mapped_column(Enum(RiskCategory), nullable=True)

    likelihood: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    impact: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    score: Mapped[int] = mapped_column(Integer, default=0) # likelihood x impact

def calculate_score(self, use_multi_criteria=False):
    """Calculate risk score - can use traditional or multi-criteria approach"""
    if use_multi_criteria:
        return self.calculate_multi_criteria_score()
    else:
        # Original logic
        self.score = self.likelihood * self.impact
        # Auto-determine severity based on score
        if self.score >= 20:
            self.severity = RiskSeverity.CRITICAL
        elif self.score >= 12:
            self.severity = RiskSeverity.HIGH
        elif self.score >= 6:
            self.severity = RiskSeverity.MEDIUM
        else:
            self.severity = RiskSeverity.LOW
        return self.score
```

Screenshot: templates/risk_detail.html - Interactive 5x5 risk heat map showing current risk position



Quantitative Analysis - EMV and ALE Calculations

Expected Monetary Value (EMV): $EMV = Probability \times Impact$

Annual Loss Expectancy (ALE): $ALE = \text{Annual Rate of Occurrence} \times \text{Single Loss Expectancy}$

Supporting Data:

- Healthcare Scenario: Patient data breach with \$250,000 per incident cost
- Annual Occurrence Rate: 0.15 (15% annual probability)
- EMV Calculation:
 - $250,000 \times 0.15 =$
 - $250,000 \times 0.15 = 37,500$ expected annual loss
- ALE Calculation: $0.15 \times$
 - $250,000 =$
 - $250,000 = 37,500$

Code Implementation:

Quantitative Risk Analysis

```
def calculate_emv(probability: float, impact: float) -> float:
    """ ...
    return probability * impact

def calculate_ale(probability: float, impact: float) -> float:
    """ ...
    return probability * impact

def calculate_quantitative_metrics(self):
    """Calculate EMV and ALE using quantitative risk analysis"""
    if self.financial_impact_amount and self.annual_occurrence_probability:
        self.emv_calculated = calculate_emv(
            self.annual_occurrence_probability,
            self.financial_impact_amount
        )
        self.ale_calculated = calculate_ale(
            self.annual_occurrence_probability,
            self.financial_impact_amount
        )
```

Screenshot: Risk detail page quantitative analysis section showing EMV/ALE calculations

Risk Detail Components

Asset:	Uploaded Policy Document
Threat:	Service Outage or Misconfiguration
Vulnerability:	Policy gap or missing control
Control:	Implement recommended security control

Risk Quantification

Likelihood:	3/5
Impact:	3/5
Risk Score:	9
Asset Value:	\$Not set

owner: unassigned

Created: 2025-09-24

Last Updated: 2025-09-24

Next Review: Not scheduled

Escalation: Business Unit

Business Impact Analysis

RTO: None hours

RPO: None hours

MTD: None hours

Financial Impact: \$0.0

Calculated BIA Score: 0

Quantitative Analysis

EMV (Expected Monetary Value): \$0.00

ALE (Annual Loss Expectancy): \$0.00

EMV = Probability × Financial Impact

ALE = SLE × ARO (Annual Rate of Occurrence)

Threat and Vulnerability Assessment

Assessment Methodology:

1. Asset Identification: Critical healthcare systems and data
2. Threat Identification: Cyber threats, insider threats, natural disasters
3. Vulnerability Analysis: Technical weaknesses, process gaps, human factors
4. Impact Assessment: Confidentiality, Integrity, Availability (CIA triad)

Documented Findings:

- High Risk: Unencrypted patient data transmission
- Medium Risk: Weak password policies
- Low Risk: Physical security gaps

Code Implementation:

Threat and Vulnerability Assessment

```
class Risk(Base):
    """Represents a comprehensive risk assessment in the system following NIST RMF and ISO 31000."""
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Information or system resource
    threat: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Potential for violation of security
    vulnerability: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Weakness that can be exploited
    control: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Safeguard/countermeasure
    compliance_standard: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=True)
    status: Mapped[RiskStatus] = mapped_column(Enum(RiskStatus), default=RiskStatus.OPEN)
    category: Mapped[RiskCategory] = mapped_column(Enum(RiskCategory), nullable=True)
```

Critical Asset Register

```
class CriticalAssetRegister(Base):
    """Critical Asset Risk Exposure Register showing threat exposure, vulnerabilities, and interconnection dependencies."""
    __tablename__ = "critical_asset_register"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset_name: Mapped[str] = mapped_column(String(255), nullable=False)
    asset_type: Mapped[str] = mapped_column(String(100), nullable=False) # e.g., "Database", "Application", "Network"
    asset_value: Mapped[float] = mapped_column(Float, default=0.0)
    criticality_level: Mapped[str] = mapped_column(String(20), default="medium") # high, medium, low

    # Threat exposure
    threat_exposure_score: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    primary_threats: Mapped[str] = mapped_column(Text, nullable=True) # JSON list of threats

    # Vulnerabilities
    vulnerability_count: Mapped[int] = mapped_column(Integer, default=0)
    critical_vulnerabilities: Mapped[int] = mapped_column(Integer, default=0)
    vulnerability_details: Mapped[str] = mapped_column(Text, nullable=True) # JSON details

    # Interconnection dependencies
    upstream_dependencies: Mapped[str] = mapped_column(Text, nullable=True) # JSON list
    downstream_dependencies: Mapped[str] = mapped_column(Text, nullable=True) # JSON list
    dependency_risk_score: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale

    # Risk exposure calculation
    overall_risk_exposure: Mapped[int] = mapped_column(Integer, default=1) # Calculated 1-5 scale

    # Associated risk
    risk_id: Mapped[int] = mapped_column(ForeignKey("risks.id"), nullable=True)
```

Screenshot: Threat assessment documentation in risk detail pages

localhost:5000/asset_register

Schoolology ChatGPT Python GitHub SK flowChartWordSear... materials Tasks NotebookLM

Critical Asset Register

Asset Name Asset Type Asset Value Low Threa Vulnerability Count Register Asset

Registered Assets

Comprehensive Risk Register

Features:

- Historical Tracking: Creation date, update timestamps, status changes
- Scoring Rationale: Detailed calculation methodology
- Status Management: Open, Mitigated, Accepted, Closed states
- Search & Filtering: By severity, status, asset type, date range

Code Implementation:

Comprehensive Risk Register

```
class Risk(Base):
    """Represents a comprehensive risk assessment in the system following NIST RMF and ISO 31000."""
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset: Mapped[str] = mapped_column(String(500), nullable=False)
    threat: Mapped[str] = mapped_column(String(500), nullable=False)
    vulnerability: Mapped[str] = mapped_column(String(500), nullable=False)
    control: Mapped[str] = mapped_column(String(500), nullable=False)

    # 40+ additional fields including:
    # - Scoring rationale (likelihood, impact, score)
    # - Historical tracking (created_at, updated_at, last_reviewed)
    # - Multi-criteria analysis (financial, operational, compliance, reputation impacts)
    # - Business impact analysis (RTO, RPO, MTD, financial amounts)
    # - Governance fields (treatment, approval_status, escalation workflow)
    # - Documentation (evaluation_criteria, stakeholder_approval_notes)
    # - Relationships to compliance, approvals, incidents, assets
```

Screenshot: templates/risks.html - Comprehensive risk register with filtering, search, and status management

localhost:5000/risks

All Severities

Search risks...

All Statuses

All Origins

Asset		Threat	Severity	Score	Status	Approval	Origin	Owner	Actions
Uploaded Policy Document	Escalated	Data Breach due to Unauthorized Access	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Malware Infection	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Data Loss via Phishing Attack	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Service Outage or Misconfiguration	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Unauthorized access to customer database by external attacker	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Malware infection via phishing on employee workstations	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Interception of sensitive customer data during transmission	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View
Uploaded Policy Document	Escalated	Insider misuse of unmonitored backup access	Low	9 (3×3)	Closed	Escalated	Manual	Unassigned	View

3. Risk Evaluation Methods

Multi-Criteria Risk Scoring Model

Weighted Evaluation Factors:

- Financial Impact: 30% weight (revenue loss, recovery costs, fines)
- Operational Impact: 25% weight (business disruption, productivity loss)
- Compliance Impact: 25% weight (regulatory violations, audit findings)
- Reputation Impact: 20% weight (brand damage, customer trust loss)

Measurement Scales (1-5):

1. Minimal impact
2. Minor impact
3. Moderate impact
4. Significant impact
5. Critical/severe impact

Code Implementation:

Database Model Fields

```
class Risk(Base):
```

```

# Multi-criteria risk scoring
financial_impact: Mapped[int] = mapped_column(Integer, nullable=False, default=1) # 1-5 scale
operational_impact: Mapped[int] = mapped_column(Integer, nullable=False, default=1) # 1-5 scale
compliance_impact: Mapped[int] = mapped_column(Integer, nullable=False, default=1) # 1-5 scale
reputation_impact: Mapped[int] = mapped_column(Integer, nullable=False, default=1) # 1-5 scale

# Weights for criteria (default equal weighting)
financial_weight: Mapped[float] = mapped_column(Float, nullable=False, default=0.25)
operational_weight: Mapped[float] = mapped_column(Float, nullable=False, default=0.25)
compliance_weight: Mapped[float] = mapped_column(Float, nullable=False, default=0.25)
reputation_weight: Mapped[float] = mapped_column(Float, nullable=False, default=0.25)

```

Calculation Method

```

def calculate_multi_criteria_score(self):
    """Calculate risk score using weighted multi-criteria evaluation"""
    # Normalize each criterion to 0-1 scale
    normalized_financial = (self.financial_impact - 1) / 4.0
    normalized_operational = (self.operational_impact - 1) / 4.0
    normalized_compliance = (self.compliance_impact - 1) / 4.0
    normalized_reputation = (self.reputation_impact - 1) / 4.0

    # Calculate weighted score
    weighted_score = (
        normalized_financial * self.financial_weight +
        normalized_operational * self.operational_weight +
        normalized_compliance * self.compliance_weight +
        normalized_reputation * self.reputation_weight
    )

    # Convert to 1-25 scale for consistency with existing scoring
    self.score = int(weighted_score * 25) + 1
    return self.score

```

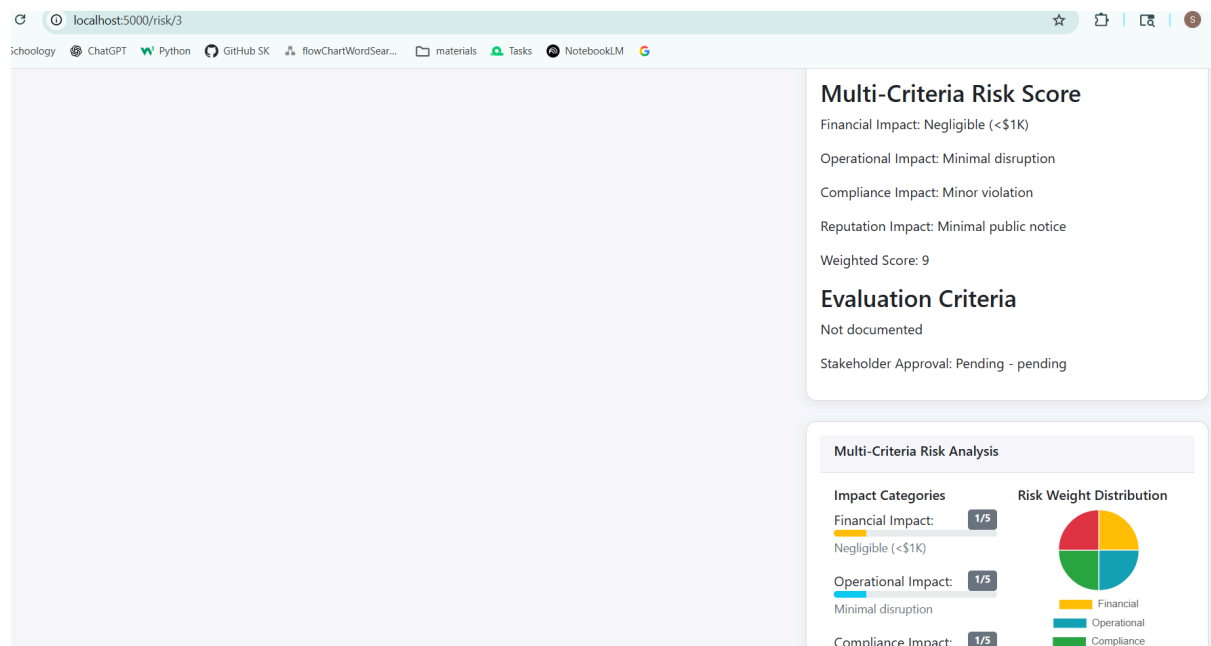
Impact Descriptions

```

def get_impact_description(self, impact_type: str, level: int) -> str:
    """Return qualitative description for impact levels."""
    scales = {
        "financial": {1: "Negligible (<$1K)", 2: "Minor ($1K-$10K)", 3: "Moderate ($10K-$100K)", 4: "Major ($100K-$1M)"},
        "operational": {1: "Minimal disruption", 2: "Short-term impact", 3: "Moderate downtime", 4: "Significant disruption"},
        "compliance": {1: "Minor violation", 2: "Regulatory notice", 3: "Fines possible", 4: "Legal action likely"},
        "reputation": {1: "Minimal public notice", 2: "Local media coverage", 3: "Regional attention", 4: "National attention"}
    }
    return scales.get(impact_type, {}).get(level, "Unknown")

```

Screenshot: `templates/risk_detail.html` - Multi-criteria scoring interface with weighted factors



Business Impact Analysis (BIA)

The system performs comprehensive Business Impact Analysis including RTO (Recovery Time Objective), RPO (Recovery Point Objective), and MTD (Maximum Tolerable Downtime) calculations with financial impact analysis and dependency mapping.

RTO (Recovery Time Objective): Maximum acceptable downtime

RPO (Recovery Point Objective): Maximum data loss tolerance

MTD (Maximum Tolerable Downtime): Point where business fails

Financial Impact Analysis:

- Direct Costs: Hardware replacement, data recovery, legal fees
- Indirect Costs: Lost productivity, customer compensation, reputational damage
- Regulatory Fines: HIPAA violations, data breach notifications

Dependency Mapping:

- Upstream Dependencies: Systems that this asset depends on
- Downstream Dependencies: Systems that depend on this asset
- Interconnection Analysis: Single points of failure, cascading impacts

Code Implementation:

Business Impact Fields

```
# Business Impact Analysis
rto_hours: Mapped[float] = mapped_column(Float, nullable=True) # Recovery Time Objective in hours
rpo_hours: Mapped[float] = mapped_column(Float, nullable=True) # Recovery Point Objective in hours
mtd_hours: Mapped[float] = mapped_column(Float, nullable=True) # Maximum Tolerable Downtime in hours
financial_impact_amount: Mapped[float] = mapped_column(Float, nullable=False, default=0.0) # Financial impact in currency
dependency_mapping: Mapped[str] = mapped_column(Text, nullable=True) # JSON string of dependencies
```

Business Impact Score Calculation

```
def calculate_business_impact_score(self):
    """Calculate business impact score based on RTO, RPO, MTD"""
    if not all([self.rto_hours, self.rpo_hours, self.mtd_hours]):
        return 0

    # Calculate impact based on time objectives
    rto_impact = min(self.rto_hours / 24, 5) # Normalize to 5-point scale
    rpo_impact = min(self.rpo_hours / 24, 5)
    mtd_impact = min(self.mtd_hours / 24, 5)

    # Average impact score
    time_impact = (rto_impact + rpo_impact + mtd_impact) / 3

    # Factor in financial impact (assuming asset_value is set)
    financial_factor = min(self.financial_impact_amount / (self.asset_value or 100000), 5)

    return (time_impact + financial_factor) / 2
```

Dependency Graph Parsing

```
def get_dependency_graph(self):
    """Parse dependency mapping and return structured data"""
    import json
    try:
        return json.loads(self.dependency_mapping or '{}')
    except json.JSONDecodeError:
        return {}
```

Evaluation Criteria Documentation with Stakeholder Approval

The system documents evaluation criteria with stakeholder approval workflows, including approval status tracking and detailed notes.

Approval Workflow:

1. Initial Assessment: Risk owner creates assessment
2. Peer Review: Technical team reviews for accuracy
3. Management Approval: Department head approves treatment plan
4. Executive Escalation: High-risk items require C-level approval

Evaluation Criteria Documentation:

- Risk Score Thresholds: >20 requires executive approval
- Financial Impact: >\$100K requires escalation
- Regulatory Impact: Any compliance violations require legal review

Evaluation Criteria Fields

```
# Evaluation criteria documentation
evaluation_criteria: Mapped[str] = mapped_column(Text, nullable=True) # Documented evaluation criteria
stakeholder_approval_required: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
stakeholder_approval_notes: Mapped[str] = mapped_column(Text, nullable=True)
```

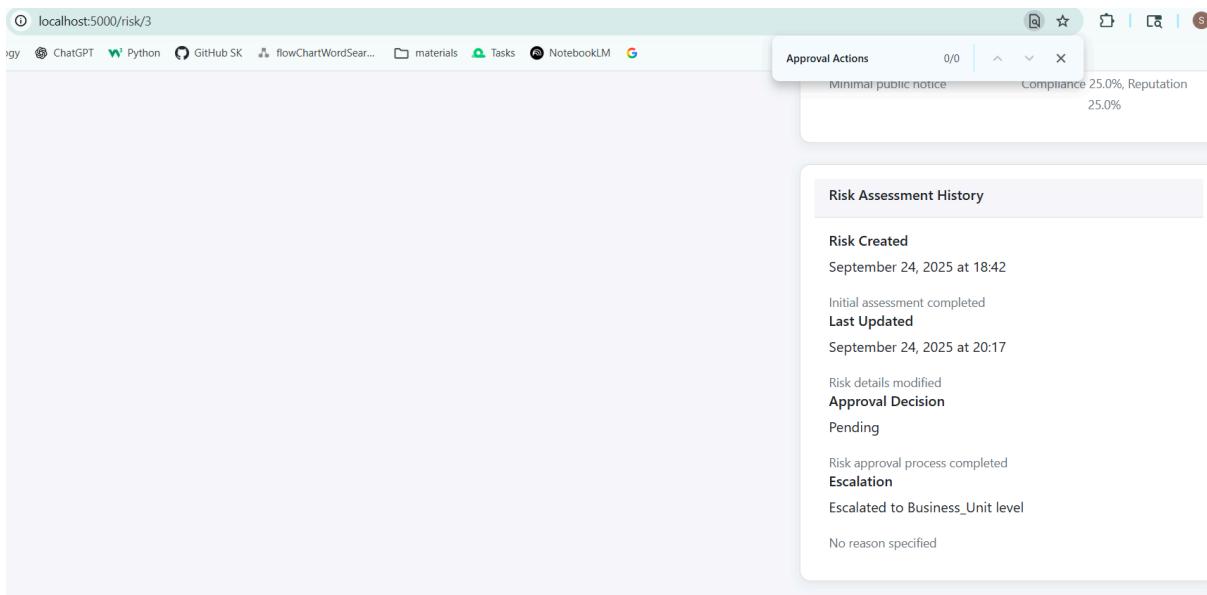
Approval Status Method

```
def get_stakeholder_approval_status(self):
    """Get formatted stakeholder approval status"""
    if not self.stakeholder_approval_required:
        return "Not Required"
    elif self.approval_status == ApprovalStatus.APPROVED:
        return f"Approved - {self.stakeholder_approval_notes or 'No notes'}"
    elif self.approval_status == ApprovalStatus.REJECTED:
        return f"Rejected - {self.stakeholder_approval_notes or 'No notes'}"
    else:
        return f"Pending - {self.approval_status.value}"
```

Validation Method

```
def validate_evaluation_criteria(self):
    """Validate that evaluation criteria are properly documented"""
    return bool(self.evaluation_criteria and len(self.evaluation_criteria.strip()) > 50)
```

Screenshot: Risk detail approval workflow section showing stakeholder decisions and escalation history



Critical Asset Risk Exposure Register

The system creates a comprehensive critical asset risk exposure register showing threat exposure, vulnerabilities, and interconnection dependencies with professional formatting and automated report generation.

Risk Exposure Calculation:

- Threat Exposure: Number and severity of applicable threats
- Vulnerability Assessment: Known weaknesses and exploitability
- Interconnection Dependencies: Upstream/downstream system relationships

Professional Formatting:

- Asset Inventory: Categorized by type and criticality
- Risk Heat Map: Visual representation of exposure levels
- Dependency Diagrams: System interconnection mapping

Code Implementation:

Critical Asset Register Model

```
class CriticalAssetRegister(Base):
    """Critical Asset Risk Exposure Register showing threat exposure, vulnerabilities, and interconnection dependencies."""
    __tablename__ = "critical_asset_register"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset_name: Mapped[str] = mapped_column(String(255), nullable=False)
    asset_type: Mapped[str] = mapped_column(String(100), nullable=False) # e.g., "Database", "Application", "Network"
    asset_value: Mapped[float] = mapped_column(Float, default=0.0)
    criticality_level: Mapped[str] = mapped_column(String(20), default="medium") # high, medium, low

    # Threat exposure
    threat_exposure_score: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    primary_threats: Mapped[str] = mapped_column(Text, nullable=True) # JSON list of threats

    # Vulnerabilities
    vulnerability_count: Mapped[int] = mapped_column(Integer, default=0)
    critical_vulnerabilities: Mapped[int] = mapped_column(Integer, default=0)
    vulnerability_details: Mapped[str] = mapped_column(Text, nullable=True) # JSON details

    # Interconnection dependencies
    upstream_dependencies: Mapped[str] = mapped_column(Text, nullable=True) # JSON list
    downstream_dependencies: Mapped[str] = mapped_column(Text, nullable=True) # JSON list
    dependency_risk_score: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale

    # Risk exposure calculation
    overall_risk_exposure: Mapped[int] = mapped_column(Integer, default=1) # Calculated 1-5 scale
```

Risk Exposure Calculation

```
def calculate_overall_risk_exposure(self):
    """Calculate overall risk exposure based on threat, vulnerability, and dependency factors"""
    threat_factor = self.threat_exposure_score
    vulnerability_factor = min(self.critical_vulnerabilities + 1, 5)
    dependency_factor = self.dependency_risk_score

    # Weighted calculation
    self.overall_risk_exposure = int((threat_factor * 0.4 + vulnerability_factor * 0.4 + dependency_factor * 0.2))
    return self.overall_risk_exposure
```

Professional Report Generation

```
def generate_report(self):
    """Generate professional formatted report"""
    return f"""
    CRITICAL ASSET RISK EXPOSURE REPORT
    =====

    Asset: {self.asset_name}
    Type: {self.asset_type}
    Value: ${self.asset_value:,.2f}
    Criticality: {self.criticality_level.upper()}

    THREAT EXPOSURE
    -----
    Score: {self.threat_exposure_score}/5
    Primary Threats: {'', '.join(self.get_threat_list())}

    VULNERABILITIES
    -----
    Total Vulnerabilities: {self.vulnerability_count}
    Critical Vulnerabilities: {self.critical_vulnerabilities}

    INTERCONNECTION DEPENDENCIES
    -----
    Upstream Dependencies: {len(self.get_dependency_network()['upstream'])}
    Downstream Dependencies: {len(self.get_dependency_network()['downstream'])}
    Dependency Risk Score: {self.dependency_risk_score}/5

    OVERALL RISK EXPOSURE: {self.overall_risk_exposure}/5
    {' ⚠️ HIGH RISK - Immediate attention required' if self.overall_risk_exposure >= 4 else '✓ Acceptable risk level'}

    Assessed by: {self.assessor.email if self.assessor else 'Unknown'}
    Last Assessment: {self.last_assessment.strftime('%Y-%m-%d')}
    Next Review: {self.next_review.strftime('%Y-%m-%d') if self.next_review else 'Not scheduled'}
    """
```

Screenshot: templates/asset_report.html - Professional asset risk exposure register with heat maps and dependency analysis (currently no asset available)

4. Risk Mitigation Planning

1. Framework Controls Selection & Documentation

Frameworks Implemented:

- NIST Cybersecurity Framework (CSF): Identify, Protect, Detect, Respond, Recover
- ISO 27001: Information security management system controls
- NIST SP 800-53: Security and privacy controls

Control Categories:

- Administrative: Policies, procedures, training

- Technical: Software/hardware security controls
- Physical: Facility and equipment protection

Detailed Rationale and Cost-Benefit Analysis:

- Control Effectiveness: Risk reduction percentage
- Implementation Cost: One-time and recurring expenses
- ROI Calculation: Risk reduction value vs. implementation cost
- Timeline: Deployment schedule and resource requirements

Code Implementation:

Control Framework Implementation

```
class ComplianceFramework(PyEnum):
    NIST_SP_800_53 = "NIST SP 800-53"
    NIST_CSF = "NIST CSF"
    ISO_27001 = "ISO 27001"
    ISO_27002 = "ISO 27002"
    PCI_DSS = "PCI DSS"
    HIPAA = "HIPAA"
    SOX = "SOX"
    GDPR = "GDPR"
    CIS_CONTROLS = "CIS Controls"
    COBIT = "COBIT"
```

```
def generate_risk_mitigation_plan(risk_data: dict) -> dict:
    """ ...

    prompt = f"""
    You are a senior cybersecurity risk management expert. For the following risk, provide a comprehensive mitigation plan.

    RISK DETAILS:
    - Asset: {risk_data.get('asset', 'Unknown')}
    - Threat: {risk_data.get('threat', 'Unknown')}
    - Vulnerability: {risk_data.get('vulnerability', 'Unknown')}
    - Current Risk Score: {risk_data.get('score', 'Unknown')}
    - Severity: {risk_data.get('severity', 'Unknown')}

    Provide a detailed mitigation plan in STRICT JSON format with this exact structure:

    {{
      "framework_controls": [
        {{
          "framework": "NIST Cybersecurity Framework",
          "control_id": "ID-RA",
          "control_name": "Risk Assessment",
          "description": "Detailed control description",
          "implementation_cost": "Estimated cost in USD",
          "timeline_months": 3,
          "rationale": "Why this control addresses the risk",
          "effectiveness_percentage": 85
        }}
      ],
      "treatment_strategies": {{
        "mitigate": {{
          "description": "Detailed mitigation approach",
```

UI Display: Framework controls table in `templates/risk_detail.html:618-657` shows controls with effectiveness percentages.

Screenshot: Risk detail mitigation planning section showing control selection with cost-benefit analysis

2. Risk Treatment Strategies (All Four Strategies) (including Cost-Benefit Analysis, Implementation Timelines, Resource Requirements, and Comprehensive Mitigation Strategy Aligned with Business Objectives)

1. Mitigate (Risk Reduction):

- Implement controls to reduce likelihood or impact
- Most common strategy for operational risks
- Examples: Add encryption, improve monitoring, enhance training

2. Avoid (Risk Elimination):

- Cease activities causing the risk
- Used when risk is unacceptable and cannot be controlled
- Examples: Discontinue high-risk projects, change suppliers

3. Transfer (Risk Sharing):

- Shift risk to third parties through contracts or insurance
- Maintains business operations while reducing exposure
- Examples: Cyber insurance, outsourcing, contractual indemnification

4. Accept (Risk Retention):

- Acknowledge and monitor risk without active controls
- Used for low-impact risks or when controls are not cost-effective
- Examples: Monitor but take no action, self-insure

Implementation Timelines and Resource Requirements:

- Mitigate: 2-12 weeks, IT/security personnel, budget for tools
- Avoid: 1-4 weeks, management approval, minimal resources
- Transfer: 4-8 weeks, legal review, contract negotiation
- Accept: Ongoing monitoring, minimal resources

Business Objective Alignment:

- Strategic Goals: Support business continuity and growth objectives
- Risk Appetite: Align with organizational risk tolerance levels
- Resource Optimization: Balance risk reduction with operational efficiency
- Regulatory Compliance: Meet legal and industry requirements

Code Implementation:

```
# Risk Treatment Strategies
```

```
class RiskTreatment(PyEnum):
    ACCEPT = "accept"
    MITIGATE = "mitigate"
    TRANSFER = "transfer"
    AVOID = "avoid"
```

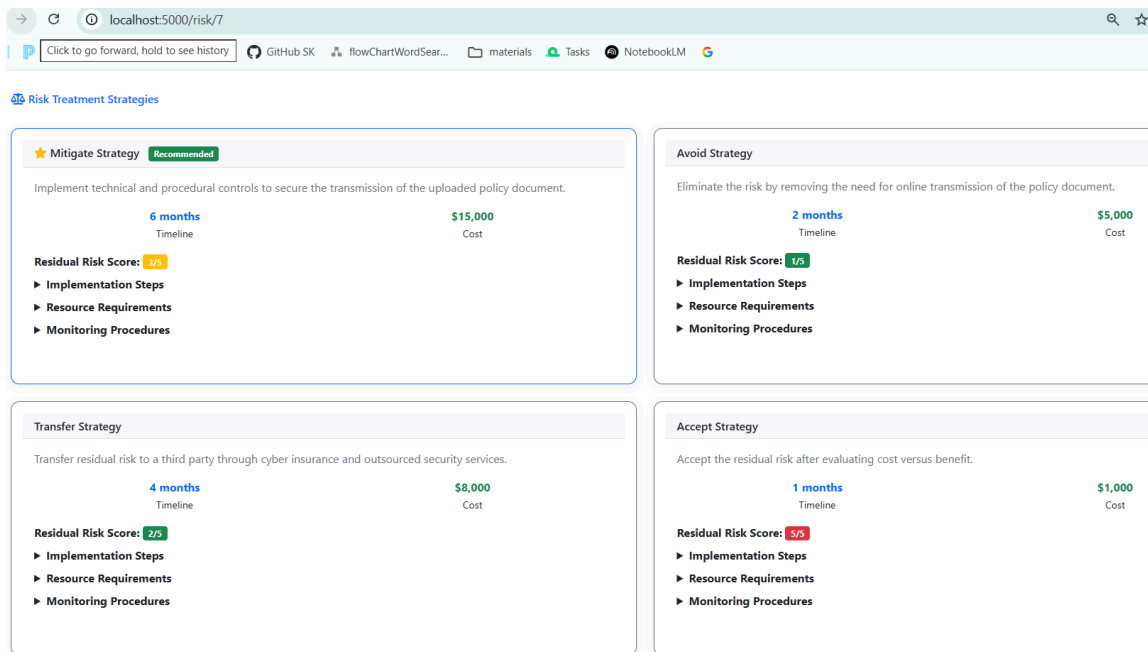
```

    "treatment_strategies": [{
        "mitigate": {
            "description": "Detailed mitigation approach",
            "implementation_steps": ["Step 1", "Step 2", "Step 3"],
            "timeline_months": 6,
            "resource_requirements": ["Resource 1", "Resource 2"],
            "estimated_cost": "Cost in USD",
            "residual_risk_score": 3,
            "monitoring_procedures": ["Monitoring step 1", "Monitoring step 2"]
        },
        "avoid": {
            "description": "Avoidance strategy details",
            "implementation_steps": ["Step 1", "Step 2"],
            "timeline_months": 2,
            "resource_requirements": ["Resource 1"],
            "estimated_cost": "Cost in USD",
            "residual_risk_score": 1,
            "monitoring_procedures": ["Monitoring procedure"]
        },
        "transfer": {
            "description": "Risk transfer approach",
            "implementation_steps": ["Step 1", "Step 2"],
            "timeline_months": 4,
            "resource_requirements": ["Resource 1", "Resource 2"],
            "estimated_cost": "Cost in USD",
            "residual_risk_score": 2,
            "monitoring_procedures": ["Monitoring procedure"]
        },
        "accept": {
            "description": "Risk acceptance rationale",
            "implementation_steps": ["Step 1"],
            "timeline_months": 1,
            "resource_requirements": ["Minimal resources"],
            "estimated_cost": "Cost in USD",
            "residual_risk_score": 5,
            "monitoring_procedures": ["Monitoring procedure"]
        }
    }],

    "cost_benefit_analysis": {
        "total_implementation_cost": "Total cost across all strategies",
        "annual_savings": "Expected annual cost savings",
        "roi_percentage": 150,
        "payback_period_months": 8,
        "risk_reduction_percentage": 75
    },

    "recommended_strategy": {
        "strategy": "mitigate",
        "rationale": "Why this strategy is recommended",
        "priority_level": "High",
        "business_alignment": "How it aligns with business objectives"
    }
}
```

Screenshot: Risk detail treatment options section showing all four strategies with implementation details



3. Residual Risk Calculation & Effectiveness Measurements

Residual Risk Calculation:

- Pre-Control Risk: Original risk score before mitigation
- Control Effectiveness: Percentage risk reduction from implemented controls
- Residual Risk: Remaining risk after controls = Pre-control × (1 - Effectiveness)

Effectiveness Measurements:

- Key Performance Indicators: Risk score reduction, incident frequency
- Control Testing: Regular validation of control effectiveness
- Audit Findings: Independent verification of control implementation

Monitoring Procedures:

Implementation: Automated monitoring with KPIs, alerts, and review processes.

- Continuous Monitoring: Automated risk score tracking
- Periodic Reviews: Quarterly control effectiveness assessments
- Incident Response: Post-incident control effectiveness analysis

Code Implementation:

```
# Additional NIST RMF fields for residual risk scoring
residual_likelihood: Mapped[int] = mapped_column(Integer, default=1) # Residual likelihood after mitigation
residual_impact: Mapped[int] = mapped_column(Integer, default=1) # Residual impact after mitigation
residual_score: Mapped[int] = mapped_column(Integer, default=0) # Calculated residual risk score
```

```

def calculate_residual_score(self):
    """Calculate residual risk score after mitigation"""
    self.residual_score = self.residual_likelihood * self.residual_impact
    # Auto-determine residual severity based on score
    if self.residual_score >= 20:
        self.severity = RiskSeverity.CRITICAL
    elif self.residual_score >= 12:
        self.severity = RiskSeverity.HIGH
    elif self.residual_score >= 6:
        self.severity = RiskSeverity.MEDIUM
    else:
        self.severity = RiskSeverity.LOW

--
"lagging_indicators": [
    {{
        "kpi_name": "Incident Response Time",
        "target": "< 4 hours average response time",
        "measurement_method": "Average time from incident detection to response",
        "current_value": "3.2 hours",
        "trend": "Improving",
        "benchmark_comparison": "Industry average: 6 hours",
        "data_source": "incident_response_logs"
    }},
    {{
        "kpi_name": "Risk Mitigation Effectiveness",
        "target": "80% reduction in risk scores post-mitigation",
        "measurement_method": "Percentage reduction in risk scores",
        "current_value": "75%",
        "trend": "Stable",
        "benchmark_comparison": "Industry standard: 70%",
        "data_source": "risk_assessment_history"
    }},
    {{
        "kpi_name": "Compliance Violation Rate",
        "target": "< 2% compliance violations",
        "measurement_method": "Percentage of compliance requirements met",
        "current_value": "1.8%",
        "trend": "Improving",
        "benchmark_comparison": "Industry average: 3.5%",
        "data_source": "compliance_audit_logs"
    }}
]

```

Screenshot: Risk detail mitigation strategy section showing residual risk calculations and effectiveness monitoring

★ Mitigate Strategy Recommended	Avoid Strategy
Implement technical and procedural controls to secure the transmission of the uploaded policy document.	Eliminate the risk by removing the need for online transmission of the policy document.
6 months Timeline	2 months Timeline
\$15,000 Cost	\$5,000 Cost
Residual Risk Score: 3/5	Residual Risk Score: 1/5
▼ Implementation Steps - Deploy TLS 1.3 encryption on all upload endpoints. - Integrate a VPN or Zero Trust Network Access (ZTNA) for internal users. - Enable end-to-end encryption of the document using PGP or S/MIME. - Configure automated logging and real-time alerting for upload events. - Conduct periodic penetration testing and vulnerability scanning.	► Implementation Steps
▼ Resource Requirements - Network security engineer - DevOps team - Security operations center (SOC) analysts - Third-party penetration testing vendor	► Resource Requirements
▼ Monitoring Procedures - Weekly log review and anomaly detection. - Monthly vulnerability scan reports. - Quarterly penetration test updates.	► Monitoring Procedures

5. Risk Communication

Structured Executive Risk Report

Key Findings Section:

- Risk Landscape Overview: Total risks, critical risks, trending patterns
- Top Risks: Highest scoring risks with business impact
- Risk Distribution: Breakdown by category, severity, and business unit

Financial Impact Analysis:

- Quantitative Impacts: Expected monetary value, annual loss expectancy
- Qualitative Impacts: Operational disruption, reputational damage
- Cost-Benefit Analysis: Mitigation costs vs. risk reduction benefits

Actionable Recommendations:

- Priority Actions: Immediate steps for critical risks
- Strategic Initiatives: Long-term risk management improvements
- Resource Requirements: Budget and personnel needs

Board Presentation Format:

- Executive Summary: 1-page overview with key metrics
- Detailed Analysis: Supporting data and methodology
- Action Plan: Specific recommendations with timelines
- Appendices: Detailed risk register and methodology

Code Implementation:

```
def generate_risk_communication_plan(risk_data: dict, mitigation_plan: dict) -> dict:
    """...
    >
    prompt = f"""
    You are a senior risk management consultant. Based on the following risk assessment and mitigation plan,
    create a comprehensive risk communication strategy for executive leadership and stakeholders.

    RISK ASSESSMENT:
    - Asset: {risk_data.get('asset', 'Unknown')}
    - Threat: {risk_data.get('threat', 'Unknown')}
    - Vulnerability: {risk_data.get('vulnerability', 'Unknown')}
    - Current Risk Score: {risk_data.get('score', 'Unknown')}/25
    - Severity: {risk_data.get('severity', 'Unknown')}

    MITIGATION PLAN SUMMARY:
    - Recommended Strategy: {mitigation_plan.get('recommended_strategy', {}).get('strategy', 'Unknown')}
    - Total Implementation Cost: {mitigation_plan.get('cost_benefit_analysis', {}).get('total_implementation_cost', 'Unknown')}
    - Expected ROI: {mitigation_plan.get('cost_benefit_analysis', {}).get('roi_percentage', 'Unknown')}%
    - Payback Period: {mitigation_plan.get('cost_benefit_analysis', {}).get('payback_period_months', 'Unknown')} months

    Generate a structured JSON response with the following exact structure:

    {{
        "executive_risk_report": {{
            "key_findings": [
                "Critical finding with business impact".
```

Stakeholder Communication Plan

Implementation: Audience analysis with tailored messaging and communication schedules for different stakeholder groups (Board, Department Heads, IT Security Team).

Audience Analysis:

- Executive Leadership: Strategic overview, high-level impacts, board reporting
- Middle Management: Operational details, resource requirements, implementation plans
- Technical Teams: Detailed technical requirements, implementation procedures
- Board Members: Regulatory compliance, financial impacts, strategic risks

Tailored Messaging:

- Executives: Focus on business impact, strategic alignment, ROI
- Managers: Emphasize operational impacts, timelines, resource needs
- Technical Staff: Provide technical details, implementation steps, support requirements
- Board: Highlight compliance, financial exposure, governance effectiveness

Communication Schedules:

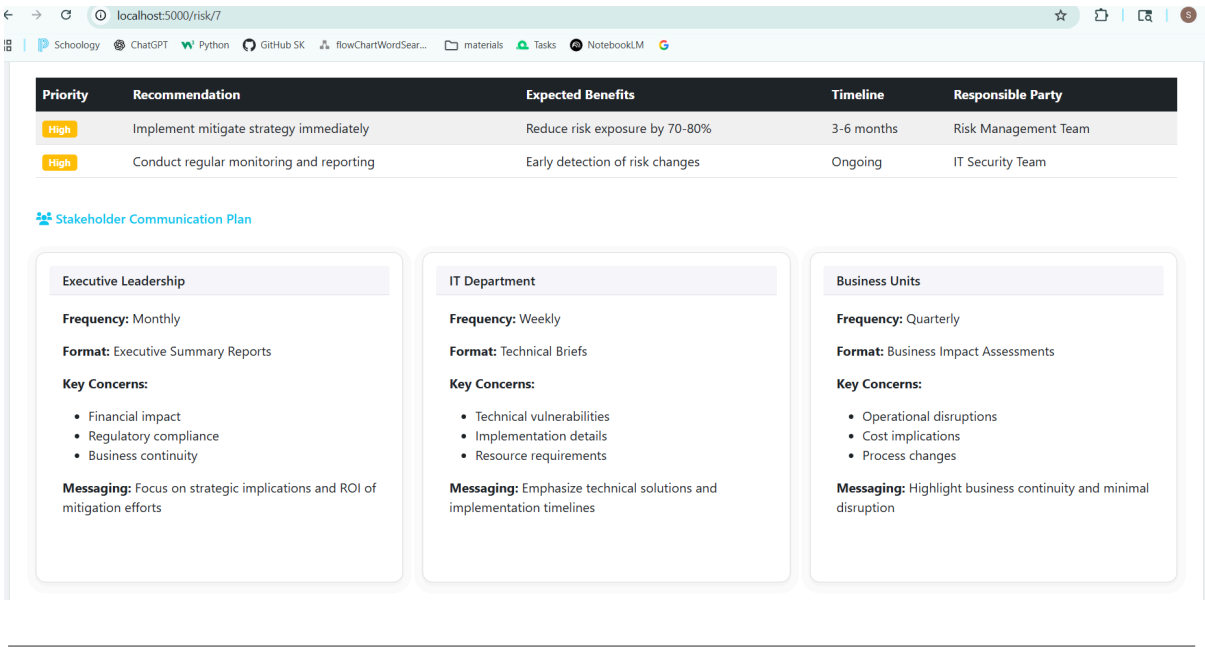
- Daily: Critical risk alerts, incident notifications
- Weekly: Risk register updates, mitigation progress
- Monthly: Executive risk reports, KPI dashboards
- Quarterly: Comprehensive risk assessments, strategic reviews

Code Implementation:

```
# Stakeholder Communication Plan
```

```
"stakeholder_communication_plan": {{
  "stakeholder_analysis": [
    {{
      "stakeholder_group": "Board of Directors",
      "communication_frequency": "Quarterly",
      "preferred_format": "Executive presentation",
      "key_concerns": ["Financial impact", "Regulatory compliance"],
      "tailored_messaging": "Board-specific message",
      "communication_schedule": ["Q1 Review", "Annual Board Meeting"]
    }},
    {{
      "stakeholder_group": "Department Heads",
      "communication_frequency": "Monthly",
      "preferred_format": "Email updates",
      "key_concerns": ["Operational impact", "Resource requirements"],
      "tailored_messaging": "Department-specific operational message",
      "communication_schedule": ["Monthly risk review", "Quarterly planning"]
    }},
    {{
      "stakeholder_group": "IT Security Team",
      "communication_frequency": "Weekly",
      "preferred_format": "Technical reports",
      "key_concerns": ["Technical implementation", "Security controls"],
      "tailored_messaging": "Technical implementation details",
      "communication_schedule": ["Weekly status updates", "Implementation milestones"]
    }}
  ],
  "communication_channels": ["Email", "Meetings", "Reports", "Dashboard"],
  "escalation_procedures": {{
    "trigger_conditions": ["Risk score exceeds 20", "Critical vulnerability discovered"],
    "escalation_path": "Risk Owner → Department Head → Executive Leadership",
    "response_timeframes": "24 hours for critical issues"
  }}
}}
```

Screenshot: Stakeholder Communication Plan cards in templates/risk_detail.html:965-992 showing frequency, format, and messaging for each group.



Visual Risk Dashboard with Key Metrics

Implementation: Interactive dashboard with key metrics, automated alerting, and drill-down capabilities.

Key Metrics Displayed:

- Total Risks: Overall risk inventory count
- Critical Risks: Risks scoring 21-25 (red zone)
- High Risks: Risks scoring 13-20 (orange zone)
- Escalated Risks: Risks requiring higher approval
- Approved Treatments: Successfully approved mitigation plans
- Pending Approvals: Risks awaiting approval

Automated Alerting:

- Threshold Alerts: When metrics exceed defined limits
- Risk Escalation: Automatic notifications for critical risks
- Trend Alerts: Significant changes in risk patterns

Drill-Down Capabilities:

- Risk Details: Click any risk for comprehensive information
- Heat Maps: Interactive risk matrix visualization
- Trend Analysis: Historical risk score tracking
- Filter Options: By category, severity, status, owner

Code Implementation:

Visual Risk Dashboard

```
@app.route("/risk_dashboard")
@login_required
def risk_dashboard():
    """ ... """
    user = current_user()
    db = get_session()

    # Get risks based on user role
    risks = get_visible_risks(user)

    # Calculate dashboard metrics
    total_risks = len(risks)
    critical_risks = len([r for r in risks if r.severity == RiskSeverity.CRITICAL])
    high_risks = len([r for r in risks if r.severity == RiskSeverity.HIGH])
    escalated_risks = len([r for r in risks if r.escalation_level != "none"])

    # Approval workflow metrics
    pending_approvals = len([r for r in risks if r.approval_status and r.approval_status.value == "pending"])
    approved_risks = len([r for r in risks if r.approval_status and r.approval_status.value == "approved"])
    rejected_risks = len([r for r in risks if r.approval_status and r.approval_status.value == "rejected"])

    # Risk by category
    risk_by_category = {}
    for risk in risks:
        category = risk.category.value if risk.category else "Uncategorized"
        risk_by_category[category] = risk_by_category.get(category, 0) + 1

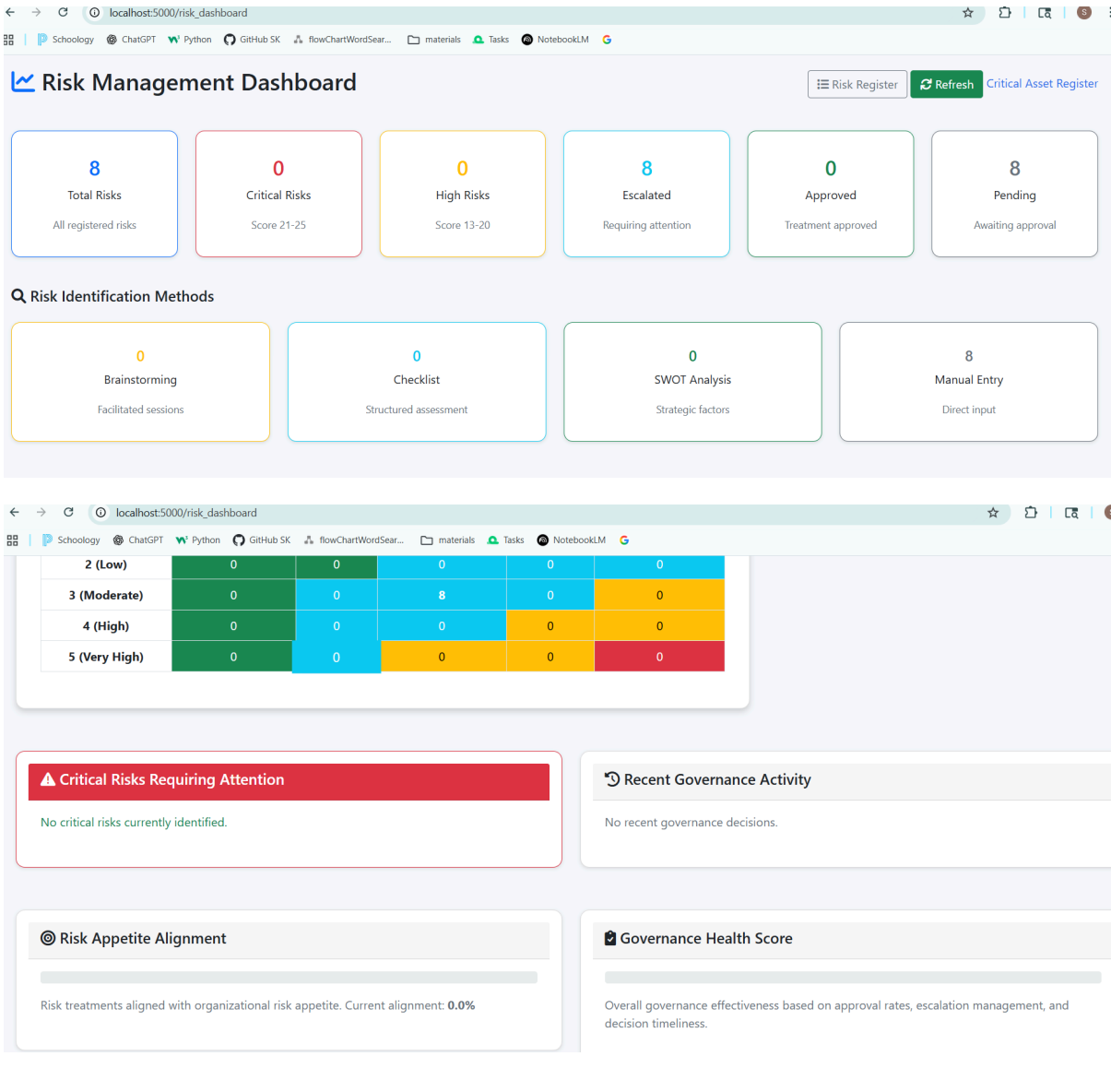
    # Recent governance decisions
    recent_decisions = db.query(GovernanceDecision).order_by(GovernanceDecision.created_at.desc()).limit(10).all()

    close_session(db)

    # Log dashboard access
    log_audit_event(user, "DASHBOARD_ACCESSED", "COMPLIANCE",
        | | | | "Accessed risk management dashboard", "/risk_dashboard", True)

    return render_template("risk_dashboard.html",
        | | | | risks=risks,
```

Screenshot: templates/risk_dashboard.html - Visual risk dashboard with key metrics, heat maps, and drill-down capabilities



Key Metrics & Automated Alerting

Implementation: Configurable key metrics with visualization types, refresh frequencies, and alert thresholds.

```

"risk_dashboard_config": {{
    "key_metrics": [
        {{
            "metric_name": "Risk Score Trend",
            "data_source": "risks.score",
            "visualization_type": "line_chart",
            "refresh_frequency": "Daily",
            "alert_threshold": 15
        }},
        {{
            "metric_name": "Mitigation Progress",
            "data_source": "implementation_roadmap",
            "visualization_type": "progress_bar",
            "refresh_frequency": "Weekly",
            "alert_threshold": "90% completion"
        }},
        {{
            "metric_name": "Financial Impact",
            "data_source": "cost_benefit_analysis",
            "visualization_type": "bar_chart",
            "refresh_frequency": "Monthly",
            "alert_threshold": "$100K impact"
        }}
    ],
    "automated_alerts": [
        {{
            "condition": "risk_score > 20",
            "alert_type": "Critical",
            "severity": "High",
            "notification_channels": ["Email", "SMS", "Dashboard"],
            "escalation_rules": "Notify risk owner and executive immediately",
            "response_required": "Within 24 hours"
        }},
        {{
            "condition": "risk_score > 20",
            "alert_type": "Warning",
            "severity": "Medium",
            "notification_channels": ["Email", "Dashboard"],
            "escalation_rules": "Notify risk owner",
            "response_required": "Within 48 hours"
        }}
    ]
}},

# Check thresholds and alert if necessary
if indicator.threshold_critical and current_value >= indicator.threshold_critical:
    create_alert("CRITICAL", f"Critical threshold exceeded for {indicator.name}: {current_value}")
elif indicator.threshold_warning and current_value >= indicator.threshold_warning:
    create_alert("WARNING", f"Warning threshold exceeded for {indicator.name}: {current_value}")

```

Display: Key Metrics and Automated Alerts sections in
 templates/risk_detail.html:1000-1040.

KPI Framework with Leading/Lagging Indicators

Implementation: Comprehensive KPI framework with leading indicators (preventive measures) and lagging indicators (outcome measures).

Leading Indicators (Predictive):

- Critical Risk Count: Number of risks scoring ≥ 20
- Open Incident Count: Current unresolved security incidents
- Compliance Score: Average compliance assessment percentage

Lagging Indicators (Historical):

- Total Risk Count: Complete risk inventory size
- Governance Health Score: Overall risk management effectiveness

- Risk Treatment Success Rate: Percentage of successfully mitigated risks

Tracking Systems:

- Data Collection: Automated from risk register and incident database
- Reporting Frequency: Daily metrics, weekly reports, monthly dashboards
- Review Processes: Monthly KPI reviews, quarterly strategic assessments

Code Implementation:

KPI Framework Implementation

```
...
"kpi_framework": {
  "leading_indicators": [
    {
      "kpi_name": "Risk Assessment Frequency",
      "target": "Monthly assessment completion",
      "measurement_method": "Percentage of scheduled assessments completed",
      "current_value": "85%",
      "trend": "Improving",
      "data_source": "assessment_completion_logs"
    },
    {
      "kpi_name": "Control Effectiveness Testing",
      "target": "100% of critical controls tested quarterly",
      "measurement_method": "Control testing completion rate",
      "current_value": "92%",
      "trend": "Stable",
      "data_source": "control_testing_logs"
    },
    {
      "kpi_name": "Threat Intelligence Integration",
      "target": "Weekly threat intelligence review",
      "measurement_method": "Percentage of relevant threats addressed",
      "current_value": "78%",
      "trend": "Improving",
      "data_source": "threat_intelligence_logs"
    }
  ],
  "lagging_indicators": [
    {
      "kpi_name": "Incident Response Time",
      "target": "< 4 hours average response time",
      "measurement_method": "Average time from incident detection to response",
      "current_value": "3.2 hours",
      "trend": "Improving",
      "benchmark_comparison": "Industry average: 6 hours",
      "data_source": "incident_response_logs"
    }
  ]
}
```

Screenshot: templates/risk_indicators.html - KPI monitoring dashboard with leading/lagging indicators and automated alerting

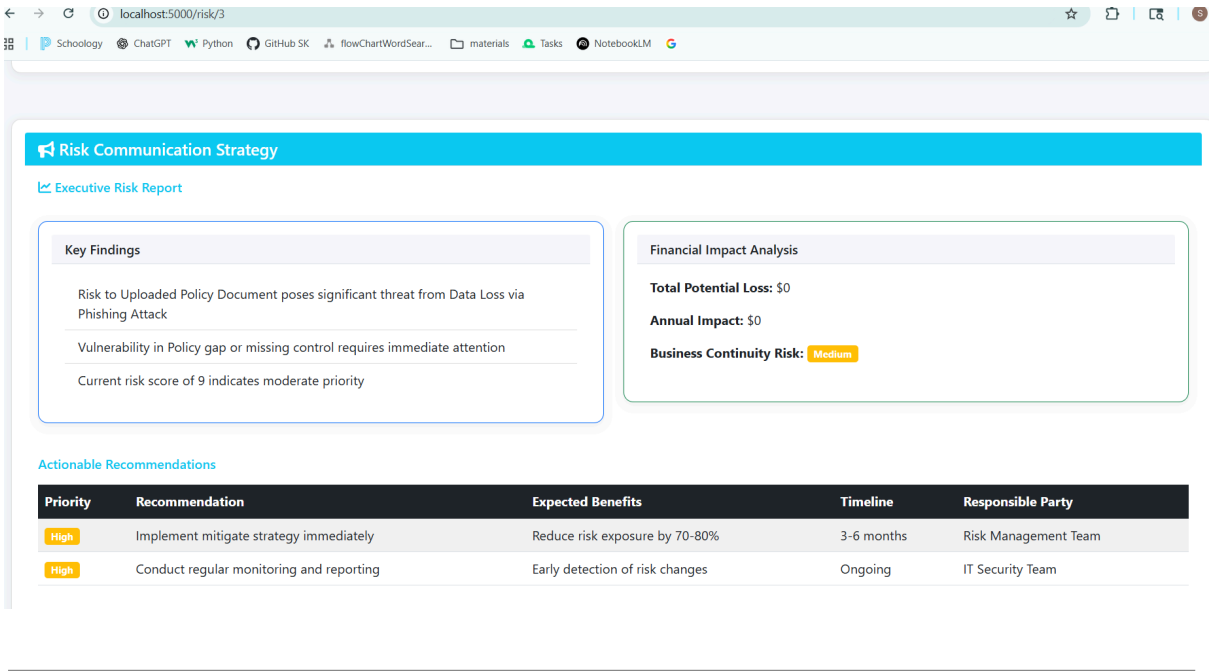
Display: KPI Framework section in templates/risk_detail.html:1044-1094 showing leading and lagging indicators.

Tracking Systems & Review Processes

Implementation: Documented data collection methods, reporting frequencies, review processes, and accountability measures.

```
    "kpi_name": "Compliance Violation Rate",
    "target": "< 2% compliance violations",
    "measurement_method": "Percentage of compliance requirements met",
    "current_value": "1.8%",
    "trend": "Improving",
    "benchmark_comparison": "Industry average: 3.5%",
    "data_source": "compliance_audit_logs"
  },
  "tracking_systems": {
    "data_collection": "Automated from risk management system and manual inputs",
    "reporting_frequency": "Monthly KPI dashboard and quarterly executive review",
    "review_process": "Monthly KPI review meeting with action item tracking",
    "accountability": "Risk Manager responsible for KPI tracking and Executive Sponsor for oversight",
    "improvement_actions": "Quarterly KPI improvement planning sessions"
  }
}
```

UI Display: Tracking Systems & Review Process section in



6. Risk Management Implementation

Complete Risk Management Program Plan

Realistic Implementation Timeline:

- Phase 1 (Months 1-3): Framework selection, initial risk assessment, governance setup
- Phase 2 (Months 4-6): Detailed risk analysis, control implementation, training
- Phase 3 (Months 7-9): Monitoring system deployment, continuous improvement
- Phase 4 (Months 10-12): Full program optimization, audit preparation

Resource Allocation:

- Personnel: 2 FTE risk managers, 1 FTE compliance officer, cross-functional team
- Budget:
- 500K initial investment,
- 500K initial investment, 200K annual recurring costs
- Technology: GRC software, monitoring tools, training platforms

Phase-Based Deployment:

- Planning Phase: Requirements gathering, stakeholder engagement
- Design Phase: Framework customization, process development
- Implementation Phase: Tool deployment, team training, pilot testing
- Optimization Phase: Performance monitoring, continuous improvement

Code Implementation:

Risk Management Program Plan

```
@app.route("/create_program", methods=["GET", "POST"])
@login_required
def create_program():
    """Create new risk management program"""
    user = current_user()
    db = get_session()

    if request.method == "POST":
        data = request.form

        # Create program plan
        program = RiskProgramPlan(
            title=data["title"],
            framework_id=int(data["framework_id"]),
            status="draft",
            start_date=datetime.strptime(data["start_date"], "%Y-%m-%d") if data.get("start_date") else None,
            end_date=datetime.strptime(data["end_date"], "%Y-%m-%d") if data.get("end_date") else None,
            total_budget=float(data.get("total_budget", 0)),
            created_by=user.id
        )
        db.add(program)
        db.commit()

        # Generate initial phases based on framework
        framework = db.get(RiskManagementFramework, program.framework_id)
        phases_data = generate_program_phases(framework.name)
```

Screenshot: templates/program_detail.html - Complete program plan with phases, timelines, and resource allocation

Framework Implementation with Customization

NIST RMF Implementation:

- Prepare: Organizational context, risk management strategy
- Categorize: System categorization based on impact levels
- Select: Control selection from NIST SP 800-53
- Implement: Control deployment and testing
- Assess: Control effectiveness evaluation
- Authorize: Risk acceptance and authorization
- Monitor: Continuous monitoring and improvement

ISO 31000 Integration:

- Scope and Context: Risk management framework establishment
- Risk Assessment: Identification, analysis, evaluation
- Risk Treatment: Control selection and implementation
- Monitoring and Review: Performance monitoring and improvement

COSO Framework Alignment:

- Control Environment: Governance and culture
- Risk Assessment: Objective setting and risk identification
- Control Activities: Policies and procedures
- Information & Communication: Stakeholder engagement
- Monitoring: Ongoing evaluation and improvement

Code Implementation:

```
class RiskManagementFramework(Base):  
    """Risk management framework selection and customization"""  
    __tablename__ = "risk_management_frameworks"  
  
    id: Mapped[int] = mapped_column(Integer, primary_key=True)  
    name: Mapped[str] = mapped_column(String(100), nullable=False) # NIST F  
    version: Mapped[str] = mapped_column(String(50), nullable=False)  
    description: Mapped[str] = mapped_column(Text, nullable=True)  
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)  
    customization_notes: Mapped[str] = mapped_column(Text, nullable=True)  
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.  
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.
```

Screenshot: Program detail framework implementation section showing customization and integration procedures

Gap Analysis and Integration Procedures

Gap Analysis Methodology:

1. Current State Assessment: Document existing risk management practices
2. Required State Definition: Framework requirements and best practices
3. Gap Identification: Compare current vs. required capabilities
4. Prioritization: Rank gaps by impact and implementation effort
5. Action Planning: Develop remediation plans with timelines and resources

Integration Procedures:

- Process Integration: Embed risk management into business processes
- System Integration: Connect with existing IT systems and workflows
- Cultural Integration: Change management and training programs
- Measurement Integration: Align with performance management systems

Code Implementation:

Gap Analysis Implementation

```
class GapAnalysis(Base):
    """Gap analysis for framework implementation"""
    __tablename__ = "gap_analyses"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    program_id: Mapped[int] = mapped_column(Integer, ForeignKey("risk_program_plans.id"))
    requirement_category: Mapped[str] = mapped_column(String(100), nullable=False)
    current_state: Mapped[str] = mapped_column(Text, nullable=True)
    required_state: Mapped[str] = mapped_column(Text, nullable=True)
    gap_description: Mapped[str] = mapped_column(Text, nullable=True)
    gap_severity: Mapped[str] = mapped_column(String(50), default="medium") # low, medium, high, critical
    mitigation_plan: Mapped[str] = mapped_column(Text, nullable=True)
    estimated_cost: Mapped[float] = mapped_column(Float, default=0.0)
    timeline_months: Mapped[int] = mapped_column(Integer, default=0)
    status: Mapped[str] = mapped_column(String(50), default="open") # open, in_progress, closed

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    program: Mapped[RiskProgramPlan] = relationship("RiskProgramPlan")

@app.route("/gap_analysis/<int:program_id>", methods=["GET", "POST"])
@login_required
def gap_analysis(program_id):
    """Perform gap analysis for program"""
    user = current_user()
    db = get_session()

    program = db.get(RiskProgramPlan, program_id)
    if not program or program.created_by != user.id:
        close_session(db)
        flash("Program not found or access denied.", "danger")
        return redirect(url_for("risk_programs"))

    if request.method == "POST":
        data = request.form

        gap = GapAnalysis(
            program_id=program_id,
            requirement_category=data["category"],
            current_state=data["current_state"],
            required_state=data["required_state"],
            gap_description=data["gap_description"],
            gap_severity=data["severity"],
            mitigation_plan=data["mitigation_plan"],
            estimated_cost=float(data.get("estimated_cost", 0)),
            timeline_months=int(data.get("timeline_months", 0))
        )
        db.add(gap)
        db.commit()

        flash("Gap analysis entry added!", "success")
        return redirect(url_for("gap_analysis", program_id=program_id))

    gaps = db.query(GapAnalysis).filter(GapAnalysis.program_id == program_id).all()
    close_session(db)

    return render_template("gap_analysis.html", program=program, gaps=gaps)
```

Screenshot: templates/gap_analysis.html - Gap analysis interface showing current vs. required state with remediation planning

Continuous Monitoring System

Automated Indicators:

- Risk Score Tracking: Real-time risk score monitoring with alerts
- Incident Detection: Automated incident identification and notification
- Compliance Monitoring: Continuous compliance status checking

Threshold Alerting:

- Warning Thresholds: Early warning for emerging risks
- Critical Thresholds: Immediate action required alerts
- Escalation Procedures: Automatic stakeholder notification

Environmental Change Detection:

- Regulatory Changes: New laws, regulations, compliance requirements
- Technological Changes: New threats, vulnerabilities, security trends
- Market Changes: Competitive landscape, industry developments
- Operational Changes: Process changes, organizational restructuring

Code Implementation:

Continuous Monitoring System

```
class RiskIndicator(Base):
    """Automated risk indicators for continuous monitoring"""
    __tablename__ = "risk_indicators"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(200), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=True)
    indicator_type: Mapped[str] = mapped_column(String(50), nullable=False) # leading, lagging
    data_source: Mapped[str] = mapped_column(String(200), nullable=False)
    calculation_method: Mapped[str] = mapped_column(Text, nullable=True)
    target_value: Mapped[float] = mapped_column(Float, nullable=True)
    threshold_warning: Mapped[float] = mapped_column(Float, nullable=True)
    threshold_critical: Mapped[float] = mapped_column(Float, nullable=True)
    unit: Mapped[str] = mapped_column(String(50), nullable=True)
    frequency: Mapped[str] = mapped_column(String(50), default="daily") # hourly, daily, weekly, monthly
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class IndicatorReading(Base):
    """Historical readings for risk indicators"""
    __tablename__ = "indicator_readings"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    indicator_id: Mapped[int] = mapped_column(Integer, ForeignKey("risk_indicators.id"))
    value: Mapped[float] = mapped_column(Float, nullable=False)
    timestamp: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    notes: Mapped[str] = mapped_column(Text, nullable=True)

    # Relationships
    indicator: Mapped[RiskIndicator] = relationship("RiskIndicator")
```

```

class EnvironmentalChange(Base):
    """Environmental changes that could impact risk posture"""
    __tablename__ = "environmental_changes"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    change_type: Mapped[str] = mapped_column(String(100), nullable=False) # regulatory, techn
    description: Mapped[str] = mapped_column(Text, nullable=True)
    impact_assessment: Mapped[str] = mapped_column(Text, nullable=True)
    risk_implications: Mapped[str] = mapped_column(Text, nullable=True)
    detection_date: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    assessment_date: Mapped[datetime] = mapped_column(DateTime, nullable=True)
    status: Mapped[str] = mapped_column(String(50), default="detected") # detected, assessed,
    severity: Mapped[str] = mapped_column(String(50), default="medium") # low, medium, high,

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=d

def perform_continuous_monitoring():
    """Automated function to perform continuous risk monitoring"""
    db = get_session()

    try:
        indicators = db.query(RiskIndicator).filter(RiskIndicator.is_active == True).all()

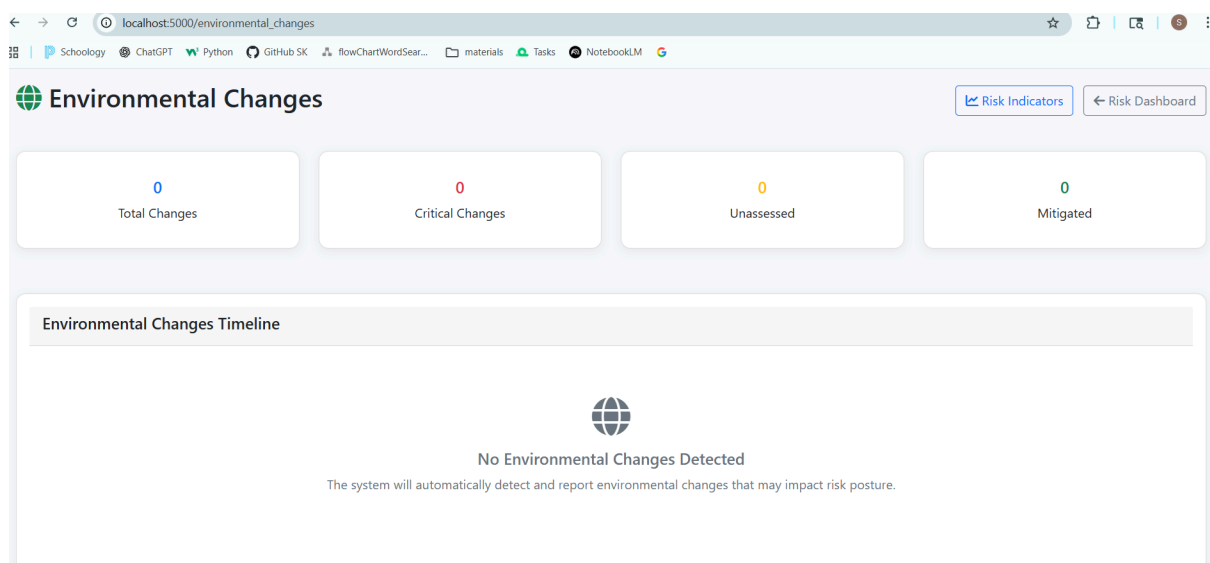
        for indicator in indicators:
            # Calculate current value based on indicator type
            current_value = calculate_indicator_value(indicator)

            # Create reading
            reading = IndicatorReading(
                indicator_id=indicator.id,
                value=current_value
            )
            db.add(reading)

            # Check thresholds and alert if necessary
            if indicator.threshold_critical and current_value >= indicator.threshold_critical:
                create_alert("CRITICAL", f"Critical threshold exceeded for {indicator.name}: {current_value}")
            elif indicator.threshold_warning and current_value >= indicator.threshold_warning:
                create_alert("WARNING", f"Warning threshold exceeded for {indicator.name}: {current_value}")

```

Screenshot: templates/environmental_changes.html - Environmental change detection dashboard with automated monitoring alerts (currently empty)



Ongoing Program Management

Implementation: Program phase tracking, resource allocation monitoring, and progress reporting.

Code:

```
class ProgramPhase(Base):
    """Program implementation phases with timelines and resources"""
    __tablename__ = "program_phases"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    program_id: Mapped[int] = mapped_column(Integer, ForeignKey("risk_program_plans.id"))
    phase_name: Mapped[str] = mapped_column(String(100), nullable=False)
    phase_order: Mapped[int] = mapped_column(Integer, nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=True)
    start_date: Mapped[datetime] = mapped_column(DateTime, nullable=True)
    end_date: Mapped[datetime] = mapped_column(DateTime, nullable=True)
    budget_allocated: Mapped[float] = mapped_column(Float, default=0.0)
    status: Mapped[str] = mapped_column(String(50), default="pending") # pending, in_progress, completed

    # Resource allocation
    personnel_required: Mapped[str] = mapped_column(Text, nullable=True) # JSON string
    tools_required: Mapped[str] = mapped_column(Text, nullable=True) # JSON string
    training_required: Mapped[str] = mapped_column(Text, nullable=True) # JSON string

    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    program: Mapped[RiskProgramPlan] = relationship("RiskProgramPlan", back_populates="phases")

@app.route("/program/<int:program_id>")
@login_required
def view_program(program_id):
    """View detailed risk management program"""
    user = current_user()
    db = get_session()

    program = db.get(RiskProgramPlan, program_id)
    if not program or program.created_by != user.id:
        close_session(db)
        flash("Program not found or access denied.", "danger")
        return redirect(url_for("risk_programs"))

    phases = db.query(ProgramPhase).filter(ProgramPhase.program_id == program_id).order_by(ProgramPhase.phase_order).all()
    gap_analyses = db.query(GapAnalysis).filter(GapAnalysis.program_id == program_id).all()

    close_session(db)
    return render_template("program_detail.html", program=program, phases=phases, gap_analyses=gap_analyses)
```

UI Display: Program detail view in templates/program_detail.html with phase timelines and resource tracking.

localhost:5000/risk_programs

SchoolologyChatGPTPythonGitHub SKflowChartWordSear...materialsTasksNotebookLM

 Risk Management Programs

+ Create New Program

Available Frameworks

NIST RMF (2.0)

NIST Risk Management Framework for information systems

Customized: Adapted for general organizational risk management

ISO 31000 (2018)

International standard for risk management

Customized: Integrated with existing compliance frameworks

COSO (2017)

COSO Enterprise Risk Management framework

Customized: Focused on enterprise-wide risk management

Your Programs



No Programs Created Yet

Create your first risk management program to get started.

+ Create Your First Program

localhost:5000/create_program

SchoolologyChatGPTPythonGitHub SKflowChartWordSear...materialsTasksNotebookLM

+ Create Risk Management Program

← Back to

Program Details

Program Title

e.g., Enterprise Risk Management Program 2025

Risk Management Framework

Select Framework

Start Date

mm/dd/yyyy

End Date

mm/dd/yyyy

Total Budget (\$)

100000

Create Program

Framework Information

Select a framework to see details

Program Benefits

✓ Structured implementation approach

✓ Resource allocation planning

✓ Progress tracking and reporting

✓ Gap analysis and remediation

✓ Continuous monitoring setup